Introduction to Reinforcement Learning

Nagesh

May 2026

"Almost everything is shallow and soon come to an end ; only to begin with further shallowness"
JK

Contents

Introduction to Reinforcement Learning

So far, we have seen machine learning models that estimate probability distributions from data. In supervised learning, we estimate conditional distributions,

$$P(y|x)$$

while in unsupervised learning, we estimate marginal distributions,

$$P(x)$$

using observed data samples.

But what happens when there is no dataset available beforehand?

To understand this, let us first think about what machine learning models are actually doing. In most learning problems, we are trying to model or simulate an environment through data samples. Once the underlying distribution is learned, we can sample from it, generate predictions, or infer hidden structure. In both supervised and unsupervised learning, the learning process fundamentally depends on a fixed dataset collected from the environment.

Reinforcement Learning (RL) addresses a very different setting.

Instead of learning passively from a pre-collected dataset, an agent actively interacts with an environment and learns through experience. The goal is not simply to model data, but to learn how to behave.

RL problems involve learning:

- what action to take,
- when to take it,
- and how to map situations to actions

in order to maximize a reward signal.

In this sense, reinforcement learning can be viewed as learning a strategy for interacting with an environment so that long-term reward becomes maximal.

Among all forms of machine learning, reinforcement learning is perhaps the closest to the way humans and animals learn. Humans do not usually learn from fixed labeled datasets. Instead, they learn by trial and error, exploration, feedback, rewards, and consequences. Many core RL algorithms were inspired by ideas from biological learning systems and behavioral psychology.

Historically, during the 1960s to 1980s, approaches based purely on search or learning were often considered "weak methods" in artificial intelligence because they lacked handcrafted reasoning and symbolic structure. Reinforcement learning represented a major shift in this perspective by demonstrating that intelligent behavior could emerge through interaction and experience.

Common Structure of RL Problems

Almost all reinforcement learning problems contain three major components:

$$\text{Agent} \rightarrow \text{Environment} \rightarrow \text{Reward}$$

- **Agent:** the decision-making system
- **Environment:** the world with which the agent interacts
- **Reward:** feedback signal that tells the agent how good or bad its action was

For example:

- In chess:
  - the player is the **agent**,
  - the chessboard and game rules form the **environment**,
  - and winning the game is the **reward**.

Unlike supervised learning, RL is not about learning a single feature or mapping from inputs to outputs. The agent must learn how the entire environment behaves and how its actions influence future states of that environment.

Thus, reinforcement learning is fundamentally both:

1. an **environment interaction problem**,
2. and a **sequential decision-making problem**.

Learning Through Interaction

A key idea in RL is that the agent improves through repeated interaction with the environment.

The learning process follows the cycle:

$$\text{Observe State} \rightarrow \text{Take Action} \rightarrow \text{Receive Reward} \rightarrow \text{Update Behavior}$$

Over time, the agent gathers experience about:

- which actions are beneficial,
- which actions are harmful,
- and which sequences of actions lead to high long-term rewards.

The more the agent interacts with the environment, the better it becomes at selecting actions that maximize future reward.

This trial-and-error mechanism is the foundation of reinforcement learning.

Elements of Reinforcement Learning Problems

There are four major elements in a reinforcement learning (RL) system:

1. Policy
2. Reward Signal
3. Value Function
4. Model of the Environment

1. Policy

Informally, a policy defines the agent's way of behaving or learning at a given time.

Formally, a policy is a mapping from perceived states of the environment to the actions taken when the agent is in those states.

Mathematically, a policy can be represented as

$$\pi(a|s)$$

which denotes the probability of taking action a when the agent is in state s.

The policy is the core of a reinforcement learning agent because it determines the behavior of the agent at every step.

For example:

- In chess, the policy determines which move to make in a given board position.
- In robotics, the policy determines which motor action to perform for a given sensor input.

The ultimate goal of RL is to learn an optimal policy that maximizes long-term reward.

2. Reward Signal

The reward signal defines the goal in a reinforcement learning problem.

At every time step, the environment sends the agent a scalar reward signal,

$$r_t \in \mathbb{R}$$

which may be either discrete or continuous.

The sole objective of the agent is to maximize the total accumulated reward over time.

Rewards are analogous to pleasure and pain in biological systems.

For example:

- If you accidentally touch a boiling vessel, your nervous system immediately generates a negative signal. This acts like a negative reward.
- If a child is promised an ice cream for remaining quiet for a minute, the child tries to maximize the chance of receiving that reward. The ice cream acts as a positive reward signal.
- Similarly, humans often seek positive social feedback. If someone sends a message and receives a warm reply, it produces a positive emotional response. If no reply comes, the experience may feel like a negative reward.

In RL, the reward signal tells the agent what is immediately good or bad, but it does not directly tell the agent what actions are best in the long run.

3. Value Function

While reward signals indicate what is good in an immediate sense, the value function specifies what is good in the long run.

The value of a state is defined as the total amount of future reward the agent can expect to accumulate starting from that state.

Mathematically, the state-value function is written as

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^{t} r_{t+1} \;\middle|\; s_0 = s\right]$$

where:

- $V^{\pi}(s)$ is the value of state s under policy π,
- γ is the discount factor,
- r_t is the reward at time step t,
- and $\mathbb{E}_{\pi}$ denotes expectation under policy π.

Consider a sports tournament with 15 matches.

Suppose a player is selected for the team but performs poorly in the first two matches. The immediate reward signal associated with the player is low.

However, the captain may still believe that the player will contribute significantly in later matches. Therefore, even though the short-term reward is low, the long-term value of the player remains high.

This illustrates the difference between reward and value:

- Reward measures immediate benefit.
- Value measures long-term desirability.

In the history of reinforcement learning, one of the most important developments between 1960 and 1990 was the realization that accurate value estimation is central to intelligent decision making.

4. Model of the Environment

A model of the environment is something that mimics the behavior of the environment.

Given the current state and action, the model predicts:

$$(\text{state}, \text{action}) \rightarrow (\text{next state}, \text{next reward})$$

More formally,

$$P(s', r \mid s, a)$$

defines the probability of transitioning to state s' and receiving reward r after taking action a in state s.

The model allows the agent to simulate future outcomes before actually taking actions.

This capability is important for planning.

For example:

- A chess engine mentally simulates future board positions before selecting a move.
- A self-driving car predicts possible future trajectories before making steering decisions.

RL methods can broadly be divided into two categories:

- **Model-Based Reinforcement Learning**: methods that explicitly learn or use a model of the environment for planning.
- **Model-Free Reinforcement Learning**: methods that learn behavior directly from experience without constructing an explicit model.

Example: Tic-Tac-Toe as a Reinforcement Learning Problem

To understand the elements of reinforcement learning more clearly, let us consider the game of Tic-Tac-Toe.

The game board consists of a 3×3 grid:

$$\begin{bmatrix} - & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$$

Suppose the RL agent plays with symbol X, while the opponent plays with symbol O.

The objective of the agent is to learn how to play the game optimally through repeated interaction with the environment.

1. Policy in Tic-Tac-Toe

The policy defines how the agent chooses actions for different board states.

A state in Tic-Tac-Toe is simply the current board configuration.

For example, consider the state:

$$\begin{bmatrix} X & O & - \\ - & X & - \\ O & - & - \end{bmatrix}$$

The possible actions are all empty positions on the board.

The policy decides which empty position the agent should place X into.

For example:

$$\pi(a|s)$$

may assign high probability to moves that help the agent win and low probability to poor moves.

Initially, the policy may behave randomly because the agent does not know how to play. After many games, the policy improves and begins selecting better actions.

Thus:

$$\text{Board State} \rightarrow \text{Policy} \rightarrow \text{Action}$$

2. Reward Signal in Tic-Tac-Toe

The reward signal tells the agent whether the outcome of its actions is good or bad.

A simple reward system can be:

$$\begin{cases} +1 & \text{if the agent wins} \\ 0 & \text{if the game is a draw} \\ -1 & \text{if the agent loses} \end{cases}$$

For example:

$$\begin{bmatrix} X & X & X \\ O & O & - \\ - & - & - \end{bmatrix}$$

Here the agent wins, so the environment gives reward:

$$r = +1$$

Similarly,

$$\begin{bmatrix} O & O & O \\ X & X & - \\ - & - & - \end{bmatrix}$$

corresponds to a loss:

$$r = -1$$

The reward signal acts as feedback that helps the agent learn which behaviors are desirable.

3. Value Function in Tic-Tac-Toe

The value function measures how good a board position is in the long run.

Consider the board:

$$\begin{bmatrix} X & O & - \\ - & X & - \\ O & - & - \end{bmatrix}$$

Even though the agent has not yet won, this position is highly valuable because the agent is close to forming a diagonal line.

Thus, the immediate reward may still be:

$$r = 0$$

but the value of the state is high because the probability of future success is large.

Another board may look like:

$$\begin{bmatrix} O & O & - \\ X & X & - \\ - & - & - \end{bmatrix}$$

Here the agent is under threat because the opponent may win in the next move. Therefore this state has lower value.

The value function estimates:

$$V(s) = \text{Expected future reward from state } s$$

Thus:

- Reward tells whether the current move was immediately good or bad.
- Value tells how promising the current board configuration is for future success.

4. Model of the Environment in Tic-Tac-Toe

The environment model predicts what happens after taking an action.

Suppose the current board is:

$$\begin{bmatrix} X & O & - \\ - & X & - \\ O & - & - \end{bmatrix}$$

If the agent places X in the bottom-right corner, the next state becomes:

$$\begin{bmatrix} X & O & - \\ - & X & - \\ O & - & X \end{bmatrix}$$

which immediately leads to victory.

Thus the model predicts:

$$(\text{current state}, \text{action}) \rightarrow (\text{next state}, \text{reward})$$

In Tic-Tac-Toe, the rules are completely known, so the environment model is easy to construct.

A model-based RL agent can simulate many future board configurations before selecting a move.

For example:

$$\text{Current Board} \rightarrow \text{Possible Future Boards} \rightarrow \text{Choose Best Action}$$

This process is called planning.

Learning Through Repeated Games

Initially, the Tic-Tac-Toe agent may play randomly and lose many games.

However, after repeatedly interacting with the environment:

- it learns which moves lead to victories,
- which moves are dangerous,
- and which board positions are strategically strong.

Over time, the agent improves its policy and eventually learns an approximately optimal strategy.

Exploration and Exploitation in Reinforcement Learning

One of the most fundamental challenges in reinforcement learning is the balance between:

$$\text{Exploration} \quad \text{vs} \quad \text{Exploitation}$$

An RL agent must constantly decide between:

- **Exploitation:** choosing actions that are already known to give high reward.

- **Exploration:** trying new actions that may produce even better rewards in the future.

This is known as the **exploration-exploitation tradeoff**.

Exploitation

Exploitation means using the current knowledge of the agent to maximize reward.

The agent selects the action that currently appears to be the best.

For example, suppose an RL agent playing Tic-Tac-Toe has learned that placing X in the center position usually increases the probability of winning.

Then the agent repeatedly chooses the center move because it already knows that the move is beneficial.

Consider the board:

$$\begin{bmatrix} - & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$$

If the agent has previously learned that the center position is strong, exploitation would result in:

$$\begin{bmatrix} - & - & - \\ - & X & - \\ - & - & - \end{bmatrix}$$

Exploitation helps the agent gain high reward using existing knowledge.

However, exploitation alone can become dangerous because the agent may never discover strategies that are even better.

Exploration

Exploration means trying actions that the agent has not sufficiently tested before.

The purpose of exploration is to gather more information about the environment.

For example, in Tic-Tac-Toe, the agent may sometimes place X in a corner position even if it currently believes the center is better:

$$\begin{bmatrix} X & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$$

Initially, this move may appear weaker, but after many games the agent may discover important long-term strategies associated with corner play.

Thus exploration allows the agent to discover:

- new strategies,

- hidden opportunities,
- and potentially better policies.

Without exploration, the agent may become trapped in a suboptimal behavior.

Why Exploration is Necessary

Suppose a child tastes only vanilla ice cream throughout life because it already gives good reward.

This is exploitation.

But unless the child explores chocolate or strawberry flavors, they may never discover something they enjoy even more.

Similarly, in RL, an agent that always exploits current knowledge may never learn the truly optimal strategy.

Therefore, good reinforcement learning algorithms carefully balance:

$$\text{Using Known Good Actions} \quad \text{and} \quad \text{Trying Unknown Actions}$$

The Exploration-Exploitation Tradeoff

The major challenge is:

- Too much exploration: the agent wastes time trying poor actions.
- Too much exploitation: the agent stops learning and may remain suboptimal.

An intelligent RL agent must balance both.

Initially, agents usually explore more because they know very little about the environment.

As learning progresses, exploration gradually decreases and exploitation increases.

Example in Tic-Tac-Toe

Suppose the agent has learned the following estimated rewards:

Move	Estimated Reward
Center	0.8
Corner	0.6
Edge	0.3

- Choosing the center repeatedly is exploitation.
- Occasionally trying a corner move to gather more information is exploration.

After many games, the agent may discover that certain corner strategies are actually better against skilled opponents.

Exploration in Biological Systems

Exploration is also common in humans and animals.

For example:

- Children explore their environment continuously.
- Animals search for new food sources even when old ones are available.
- Humans try new ideas, careers, foods, and relationships despite uncertainty.

This balance between exploiting known rewards and exploring unknown possibilities is a fundamental characteristic of intelligent behavior.

Exploitation	**Exploration**
Uses existing knowledge	Gathers new knowledge
Chooses best known action	Tries uncertain actions
Maximizes immediate reward	May improve future reward
Low uncertainty	High uncertainty

A successful reinforcement learning agent must continuously balance exploration and exploitation in order to learn an optimal policy.

Multi-Armed Bandit Problem

The **Multi-Armed Bandit** problem is one of the simplest and most important problems in reinforcement learning.

It captures the fundamental challenge of:

$$\text{Exploration} \quad \text{vs} \quad \text{Exploitation}$$

The name comes from gambling machines.

A slot machine is often called a "one-armed bandit" because it has a single lever (arm) and can steal your money.

Now imagine a casino with multiple slot machines:

$$\text{Machine 1, Machine 2, Machine 3, } \ldots, \text{ Machine } k$$

Each machine gives rewards according to some unknown probability distribution.

The problem is:

> How should a player choose machines over time in order to maximize total reward?

Basic Setup

Suppose there are k slot machines.

At every time step:

- the agent selects one machine,
- the machine gives a reward,
- and the agent learns from the received reward.

Each machine is called an **arm**.

Thus the problem is called a:

$$k\text{-armed bandit problem}$$

Mathematical Representation

Suppose there are k actions:

$$a_1, a_2, a_3, \ldots, a_k$$

Each action has an unknown true value:

$$q_*(a) = \mathbb{E}[R_t | A_t = a]$$

where:

- $q_*(a)$ is the true expected reward of action a,
- R_t is the reward at time t,
- A_t is the action selected at time t.

The goal of the agent is to estimate these unknown values and choose the best arm as often as possible.

Real-Life Example

Suppose a food delivery company wants to decide which advertisement to show users.

There are three advertisements:

- Ad A
- Ad B
- Ad C

Each advertisement has an unknown probability of user clicks.

The company wants to maximize total clicks.

If the company always shows Ad A because it initially performed well, it may miss discovering that Ad C is actually better.

Thus the company faces the exploration-exploitation dilemma.

Exploration vs Exploitation in Bandits

Exploitation

Exploitation means choosing the arm that currently appears best.

Suppose after some trials the estimated rewards are:

Machine	Estimated Reward
A	5.2
B	2.1
C	4.8

Exploitation chooses:

$$A$$

because it currently gives the highest estimated reward.

This maximizes immediate gain.

Exploration

Exploration means trying other machines even if they currently appear worse.

For example:

- Maybe Machine B was tested only twice.
- Maybe Machine C becomes better over long runs.

Thus the agent occasionally selects less-tested machines to gather more information.

Exploration may temporarily reduce reward, but it helps discover potentially better actions.

The Core Tradeoff

The central challenge is:

$$\text{Should the agent exploit known good actions?}$$

or

$$\text{Should the agent explore uncertain actions?}$$

Too much exploitation can prevent discovery of better actions.

Too much exploration wastes time on poor actions.

Good RL algorithms balance both.

Estimating Action Values

The agent estimates the value of each arm using the average observed reward.

Suppose action a has been selected $N(a)$ times.

Then the estimated value is:

$$Q(a) = \frac{\sum_{i=1}^{N(a)} R_i}{N(a)}$$

where:

- $Q(a)$ is the estimated value,
- R_i are observed rewards from that arm.

Over time:

$$Q(a) \to q_*(a)$$

meaning the estimate approaches the true expected reward.

Example of Learning

Suppose an agent interacts with three machines.

Machine	Observed Rewards
A	$5, 4, 6, 5$
B	$1, 2$
C	$3, 8, 4$

Estimated values become:

$$Q(A) = \frac{5+4+6+5}{4} = 5$$

$$Q(B) = \frac{1+2}{2} = 1.5$$

$$Q(C) = \frac{3+8+4}{3} = 5$$

Now the agent believes both A and C are good choices.

ϵ-Greedy Strategy

One common strategy is the ϵ-greedy method.

The rule is:

- With probability $1 - \epsilon$: exploit the best known action.
- With probability ϵ: explore a random action.

For example:

$$\epsilon = 0.1$$

means:

- 90% of the time the agent exploits,
- 10% of the time the agent explores.

This simple strategy works surprisingly well in many problems.

Importance of Multi-Armed Bandits

The multi-armed bandit problem is important because it forms the foundation of many RL ideas.

Applications include:

- recommendation systems,
- online advertising,
- medical treatment selection,
- robotics,
- finance,
- game playing,
- and adaptive optimization systems.

Although simple, the bandit problem beautifully illustrates how intelligent systems learn through trial, feedback, exploration, and exploitation.

Finite Markov Decision Processes

Most reinforcement learning problems can be formally modeled using a **Markov Decision Process (MDP)**.

An MDP provides the mathematical framework for describing:

- the environment,
- the agent,
- actions,
- rewards,
- and state transitions.

When the number of states and actions are finite, the problem is called a **Finite Markov Decision Process**.

Basic Components of an MDP

A finite MDP consists of:

$$(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$$

where:

- $\mathcal{S}$: finite set of states
- $\mathcal{A}$: finite set of actions
- $\mathcal{P}$: state transition probabilities
- $\mathcal{R}$: reward function
- γ : discount factor

States

A **state** represents the current situation of the environment.

For example:

- In chess, a state is the current board configuration.
- In Tic-Tac-Toe, a state is the arrangement of X and O.
- In robotics, a state may include position, velocity, and sensor values.

The set of all possible states is denoted by:

$$\mathcal{S} = \{s_1, s_2, s_3, \dots\}$$

Actions

An **action** is a decision the agent can take.

The action set is:

$$\mathcal{A} = \{a_1, a_2, a_3, \dots\}$$

Examples:

- In chess: move a piece
- In driving: accelerate, brake, turn
- In Tic-Tac-Toe: place X in an empty cell

At every time step t:

$$A_t \in \mathcal{A}(S_t)$$

meaning the action depends on the current state.

Rewards

After taking an action, the environment gives a reward:

$$R_{t+1} \in \mathbb{R}$$

The reward measures the immediate desirability of the action.
Examples:

- Winning a game: positive reward
- Losing a game: negative reward
- Crashing a robot: negative reward

The objective of the agent is:

$$\text{maximize expected cumulative reward}$$

State Transition Probabilities

The environment changes from one state to another after an action is taken.
This is described using transition probabilities:

$$p(s', r \mid s, a)$$

which means:

> Probability that the next state becomes s' and reward r is received, given that the current state is s and action a was taken.

Mathematically:

$$p(s', r|s, a) = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)$$

These probabilities completely characterize the environment dynamics.

Markov Property

The defining feature of an MDP is the **Markov Property**.
It states that:

> The future depends only on the present state and action, not on the entire past history.

Mathematically:

$$\Pr(S_{t+1}|S_t) = \Pr(S_{t+1}|S_1, S_2, \ldots, S_t)$$

This means the current state contains all relevant information needed for future prediction.

Example: Tic-Tac-Toe as an MDP

Consider Tic-Tac-Toe.

States

Each board configuration is a state.

Example:

$$s = \begin{bmatrix} X & O & - \\ - & X & - \\ O & - & - \end{bmatrix}$$

Actions

Possible actions are empty cells where X can be placed.

For the above board:

$$\mathcal{A}(s) = \{(1,3), (2,1), (2,3), (3,2), (3,3)\}$$

Rewards

$$R = \begin{cases} +1 & \text{win} \\ 0 & \text{draw} \\ -1 & \text{loss} \end{cases}$$

Transitions

If the agent places X at $(3,3)$:

$$\begin{bmatrix} X & O & - \\ - & X & - \\ O & - & X \end{bmatrix}$$

the game ends with reward:

$$R = +1$$

Example: Robot Navigation

Suppose a robot moves inside a grid world.

$$\begin{matrix} S_1 & S_2 & S_3 \\ S_4 & S_5 & S_6 \\ S_7 & S_8 & G \end{matrix}$$

where G is the goal state.

States

Each grid position is a state.

Actions

Possible actions:

$$\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}$$

Rewards

$$R = \begin{cases} +10 & \text{reach goal} \\ -1 & \text{each step} \end{cases}$$

Transitions

If the robot is in S_5 and moves right:

$$S_5 \to S_6$$

The environment may also be stochastic:

$$P(S_6|S_5, \text{right}) = 0.8$$

$$P(S_2|S_5, \text{right}) = 0.1$$

$$P(S_8|S_5, \text{right}) = 0.1$$

meaning the robot sometimes slips.

Episodes

Many RL tasks are divided into episodes.

An episode consists of:

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_T$$

where:

- S_0 is the starting state,
- S_T is the terminal state.

Examples:

- one chess game,
- one Tic-Tac-Toe game,
- one robot navigation trial.

Discounted Return

The agent tries to maximize the total future reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

or compactly:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where:

- G_t is called the return,
- $\gamma \in [0, 1]$ is the discount factor.

Interpretation of the Discount Factor

$$\gamma \approx 0$$

means:

- the agent cares mostly about immediate rewards.

$$\gamma \approx 1$$

means:

- the agent values long-term rewards strongly.

A finite Markov Decision Process provides a complete mathematical framework for reinforcement learning problems.

An MDP contains:

Component	Meaning
State S_t	Current situation
Action A_t	Decision taken by agent
Reward R_t	Immediate feedback
Transition Probability	Environment dynamics
Policy $\pi(a\|s)$	Agent behavior
Return G_t	Total future reward

Most modern reinforcement learning algorithms are built upon the mathematical foundation of Markov Decision Processes.

Episodic and Continuous Tasks in Reinforcement Learning

Reinforcement learning problems are generally divided into two major categories:

$$\text{Episodic Tasks} \quad \text{and} \quad \text{Continuous Tasks}$$

The distinction depends on whether the interaction between the agent and the environment eventually terminates.

Episodic Tasks

An **episodic task** is a task that has a clear beginning and a terminal ending state.

The agent interacts with the environment for a finite sequence of steps called an **episode**.

After reaching the terminal state, the environment resets and a new episode begins.

The interaction sequence looks like:

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_T$$

where:

$$S_T$$

is the terminal state.

Characteristics of Episodic Tasks

- There is a definite start state.
- The task eventually terminates.
- Rewards are accumulated only within an episode.
- After termination, the environment resets.

Example: Tic-Tac-Toe

Consider Tic-Tac-Toe.

The game starts with an empty board:

$$\begin{bmatrix} - & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$$

The agent and opponent alternately place symbols until:

- someone wins,
- or the game becomes a draw.

At that point the episode ends.
For example:

$$\begin{bmatrix} X & X & X \\ O & O & - \\ - & - & - \end{bmatrix}$$

Here the agent wins:

$$R = +1$$

and the game terminates.
The next game begins again from an empty board.
Thus:

$$\text{One Complete Game} = \text{One Episode}$$

Other examples of episodic tasks include:

- Chess
- Maze solving
- Playing a video game level
- Robotic object pickup

Continuous Tasks

A **continuous task** is a task that does not naturally terminate.

The agent continuously interacts with the environment forever or for a very long duration.

There is no terminal state.

The interaction sequence becomes:

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots$$

without an endpoint.

Characteristics of Continuous Tasks

- No natural termination state.
- The agent must continuously make decisions.
- Long-term behavior becomes extremely important.
- The objective is often maintaining stable performance indefinitely.

Example: Self-Driving Car

Consider a self-driving car.
The car continuously observes:

- road conditions,
- nearby vehicles,
- traffic signals,
- pedestrians.

The agent continuously performs actions:

$$\{\text{accelerate}, \text{brake}, \text{turn}\}$$

The task usually has no natural ending.
The car must keep driving safely for long periods.
Rewards may include:

$$\begin{cases} +1 & \text{safe driving} \\ -100 & \text{collision} \\ -1 & \text{traffic violation} \end{cases}$$

Unlike Tic-Tac-Toe, the environment does not reset after every few steps.
The interaction continues indefinitely.
Other examples include:

- Stock trading systems
- Industrial process control
- Power grid optimization
- Robot balancing
- Recommendation systems

Comparison Between Episodic and Continuous Tasks

Feature	Episodic Task	Continuous Task
Termination	Has terminal state	No terminal state
Interaction Length	Finite	Potentially infinite
Environment Reset	Yes	Usually no
Examples	Chess, Tic-Tac-Toe	Driving, Trading
Focus	Winning episode	Sustained long-term behavior

Return in Episodic Tasks

In episodic tasks, the return is finite because the episode terminates:

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$$

The summation naturally stops at the terminal state.

Return in Continuous Tasks

In continuous tasks, the return may continue forever:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The discount factor:

$$0 \leq \gamma < 1$$

ensures that the infinite sum remains finite.

Importance of Discounting in Continuous Tasks

In continuous environments, future rewards may extend infinitely far into the future.

Without discounting:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \ldots$$

may diverge.

The discount factor reduces the contribution of distant rewards:

$$\gamma^k$$

thus ensuring mathematical stability.

Intuition

- Episodic tasks are like playing separate matches in a tournament.
- Continuous tasks are like living daily life where decisions never truly stop.

In episodic tasks, the goal is often:

maximize success before termination

In continuous tasks, the goal is:

maintain optimal behavior indefinitely

Task Type	Example
Episodic	Tic-Tac-Toe, Chess, Maze
Continuous	Driving, Trading, Robotics Control

Understanding the distinction between episodic and continuous tasks is important because different RL algorithms and value estimation methods are often designed specifically for one setting or the other.

Formulating a Markov Decision Process (MDP)

Let us formulate a simple reinforcement learning problem as a Markov Decision Process (MDP).

Consider a robot navigating in a grid world.

The robot must move from the start position to the goal position while avoiding obstacles.

Grid World Environment

Suppose the environment is:

$$\begin{array}{ccc} S_1 & S_2 & S_3 \\ S_4 & S_5 & S_6 \\ S_7 & S_8 & G \end{array}$$

where:

- S_1 is the start state,
- G is the goal state.

The agent can move:

$$\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}$$

MDP Formulation

The MDP is represented as:

$$(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$$

1. States

The finite state space is:

$$\mathcal{S} = \{S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, G\}$$

Each state represents the current position of the robot.

2. Actions

The action set is:

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$$

These correspond to moving:

- up,
- down,
- left,
- right.

3. Transition Probabilities

Suppose the environment is stochastic.

If the robot attempts to move right from S_5:

$$P(S_6|S_5, \rightarrow) = 0.8$$

$$P(S_2|S_5, \rightarrow) = 0.1$$

$$P(S_8|S_5, \rightarrow) = 0.1$$

meaning:

- 80% probability of moving correctly,
- 10% probability of slipping upward,
- 10% probability of slipping downward.

4. Reward Function

The reward function is:

$$R(s, a, s') = \begin{cases} +10 & \text{if } s' = G \\ -1 & \text{otherwise} \end{cases}$$

Thus:

- reaching the goal gives positive reward,
- every movement has a small negative cost.

The step penalty encourages shorter paths.

5. Discount Factor

Suppose:

$$\gamma = 0.9$$

This means the agent values future rewards strongly but still slightly prefers immediate rewards.

Objective of the Agent

The objective is to learn a policy:

$$\pi(a|s)$$

that maximizes expected return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The optimal policy learns the shortest and safest path toward the goal.

Backup Diagram

A backup diagram visually represents how value information propagates through states in reinforcement learning.

Suppose the robot is currently in state:

$$S_t = S_5$$

and can take several actions.

One-Step Backup Diagram

$$\begin{array}{ccc} & V(S_5) & \\ & \downarrow & \\ a_1 & a_2 & a_3 \\ \downarrow & \downarrow & \downarrow \\ S_2 & S_6 & S_8 \end{array}$$

This diagram means:

- the agent evaluates possible actions,
- each action leads probabilistically to next states,
- future state values are backed up to update the current state value.

Bellman Backup Equation

The value backup is mathematically written as:

$$V(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma V(s') \right]$$

This equation states:

> The value of the current state equals the expected immediate reward plus the discounted value of future states.

Backup Intuition

Suppose:

$$S_6$$

is very close to the goal and has high value.
Then:

$$V(S_6)$$

propagates backward and increases:

$$V(S_5)$$

This process of propagating future rewards backward through states is called a:

backup

and it is one of the central ideas in dynamic programming and reinforcement learning.

Example of Value Propagation

Suppose:

$$V(S_6) = 8$$

and moving from S_5 to S_6 gives reward:

$$R = -1$$

Then:

$$V(S_5) \approx -1 + 0.9(8) = 6.2$$

Thus the high value near the goal propagates backward through the environment.

A Markov Decision Process formally describes reinforcement learning problems using:

- states,
- actions,
- rewards,
- transition probabilities,
- and discounting.

Backup diagrams illustrate how future rewards propagate backward through states during learning and planning.

Value Functions in Reinforcement Learning

One of the most important ideas in reinforcement learning is the concept of a **value function**.

A value function tells us:

> "How good is a particular state or action in the long run?"

While rewards measure immediate benefit, value functions measure expected future reward.

The entire objective of reinforcement learning is fundamentally based on estimating and optimizing value functions.

Return

Before defining value functions, we first define the concept of **return**.

The return at time step t is the total future accumulated reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

or compactly:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where:

- R_t is the reward,
- $\gamma \in [0, 1]$ is the discount factor.

State-Value Function

The **state-value function** measures how good it is for the agent to be in a particular state under a policy π.

It is denoted as:

$$V^{\pi}(s)$$

and defined as:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

This means:

> Expected future return when starting from state s and following policy π.

Interpretation

Suppose a robot is navigating toward a goal.

- States near the goal have high value.
- States near obstacles or traps have low value.

Thus:

$$V^{\pi}(s)$$

represents the long-term desirability of state s.

Action-Value Function

The **action-value function** measures how good it is to take a particular action in a given state under policy π.

It is denoted by:

$$Q^{\pi}(s, a)$$

and defined as:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

This means:

> Expected return after taking action a in state s, and then continuing according to policy π.

Difference Between State and Action Value

$$V^{\pi}(s)$$

asks:

"How good is this state?"

while

$$Q^{\pi}(s, a)$$

asks:

"How good is taking this specific action in this state?"

Bellman Equation for State-Value Function

The Bellman equation expresses a recursive relationship for value functions.
Starting from:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

Using:

$$G_t = R_{t+1} + \gamma G_{t+1}$$

we get:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

Using linearity of expectation:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V^{\pi}(S_{t+1}) \mid S_t = s]$$

Expanding over actions and transitions:

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s', r} p(s', r|s, a) \left[r + \gamma V^{\pi}(s')\right]$$

This is the **Bellman Expectation Equation**.

Bellman Equation for Action-Value Function

Similarly:

$$Q^\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$$

which becomes:

$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a)\,[r + \gamma V^\pi(s')]$$

Since:

$$V^\pi(s') = \sum_{a'} \pi(a'|s') Q^\pi(s',a')$$

we obtain:

$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a') \right]$$

Optimal Value Functions

The goal of RL is to find the optimal policy:

$$\pi^*$$

that maximizes expected return.

Optimal State-Value Function

The optimal state-value function is:

$$V^*(s) = \max_\pi V^\pi(s)$$

It gives the maximum achievable expected return from state s.

Optimal Action-Value Function

Similarly:

$$Q^*(s,a) = \max_\pi Q^\pi(s,a)$$

Bellman Optimality Equation

The Bellman optimality equation for state values is:

$$V^*(s) = \max_a \sum_{s',r} p(s', r|s, a) \left[r + \gamma V^*(s')\right]$$

This equation says:

> The optimal value of a state equals the best possible expected future return obtainable from any action.

Bellman Optimality for Action Values

Similarly:

$$Q^*(s, a) = \sum_{s',r} p(s', r|s, a) \left[r + \gamma \max_{a'} Q^*(s', a')\right]$$

Optimality Condition

Once we know the optimal action-value function:

$$Q^*(s, a)$$

the optimal policy is simply:

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

meaning:

> Choose the action with maximum action value.

Worked Example

Consider a simple environment:

$$S_1 \to S_2 \to G$$

Suppose:

- moving from S_1 to S_2 gives reward:

$$R = -1$$

- moving from S_2 to goal gives reward:

$$R = 10$$

- discount factor:

$$\gamma = 0.9$$

Step 1: Goal State Value

Since the goal is terminal:

$$V(G) = 0$$

Step 2: Value of S_2

Using Bellman equation:

$$V(S_2) = 10 + 0.9(0)$$

thus:

$$V(S_2) = 10$$

Step 3: Value of S_1

Now:

$$V(S_1) = -1 + 0.9(10)$$

$$V(S_1) = 8$$

Thus:

$$V(S_1) = 8, \qquad V(S_2) = 10$$

Interpretation

Notice:

- S_1 has lower value because the reward is delayed.
- S_2 has higher value because it is closer to the goal.

This demonstrates how future rewards propagate backward through states.

Backup Diagram

$$
\begin{array}{c}
V(S_1) \\
\downarrow \\
R = -1 \\
\downarrow \\
V(S_2) \\
\downarrow \\
R = 10 \\
\downarrow \\
G
\end{array}
$$

The value information flows backward from the goal toward earlier states.

Concept	Meaning
$V^{\pi}(s)$	Value of a state
$Q^{\pi}(s, a)$	Value of taking action a in state s
Bellman Equation	Recursive value relation
$V^{*}(s)$	Optimal state value
$Q^{*}(s, a)$	Optimal action value
Optimal Policy	Choose action with max $Q^{*}(s, a)$

Value functions form the mathematical foundation of almost all reinforcement learning algorithms.

Derivation of the Bellman Equation from Scratch

The Bellman equation is one of the most fundamental equations in reinforcement learning.

It provides a recursive relationship between:

- the value of the current state,
- the immediate reward,
- and the value of future states.

The central idea is:

> The value of a state equals the immediate reward plus the value of future states.

Step 1: Define the Return

Suppose an agent interacts with an environment over time.

At each time step:

$$S_t \to A_t \to R_{t+1}$$

The agent receives rewards sequentially:

$$R_{t+1}, R_{t+2}, R_{t+3}, \ldots$$

The total future accumulated reward is called the **return**.
It is defined as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$$

or compactly:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where:

- G_t is the return,
- R_t is the reward,
- $\gamma \in [0, 1]$ is the discount factor.

Step 2: Recursive Nature of Return

Now observe carefully:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots$$

We can separate the first reward term:

$$G_t = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \ldots)$$

But the expression inside parentheses is exactly:

$$G_{t+1}$$

Therefore:

$$G_t = R_{t+1} + \gamma G_{t+1}$$

This recursive decomposition is the key idea behind the Bellman equation.

Step 3: Define State-Value Function

The state-value function under policy π is defined as:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

meaning:

Expected return when starting from state s and following policy π.

Step 4: Substitute Recursive Return

Using:

$$G_t = R_{t+1} + \gamma G_{t+1}$$

we substitute into the value function:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

Using linearity of expectation:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_t = s]$$

But:

$$\mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s'] = V^{\pi}(s')$$

Thus:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V^{\pi}(S_{t+1}) \mid S_t = s]$$

This is the Bellman expectation equation in expectation form.

Step 5: Expand Using Transition Probabilities

Now we expand the expectation explicitly.

The policy determines actions:

$$\pi(a|s)$$

The environment determines transitions:

$$p(s', r|s, a)$$

Therefore:

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s', r} p(s', r|s, a) \left[r + \gamma V^{\pi}(s') \right]$$

This is the full Bellman expectation equation.

Interpretation of the Equation

The equation says:

$$\boxed{\text{Current Value} = \text{Immediate Reward} + \text{Discounted Future Value}}$$

The value of a state depends on:

- the actions selected by the policy,
- the rewards received,
- and the values of future states.

Bellman Equation for Action-Value Function

Now define the action-value function:

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Substitute the recursive return:

$$Q^\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$$

Replacing future return by future value:

$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a)\,[r + \gamma V^\pi(s')]$$

Since:

$$V^\pi(s') = \sum_{a'} \pi(a'|s') Q^\pi(s',a')$$

we obtain:

$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a') \right]$$

Bellman Optimality Equation

The optimal value function is:

$$V^*(s) = \max_\pi V^\pi(s)$$

The optimal policy always chooses the action giving maximum future value. Thus:

$$V^*(s) = \max_a \sum_{s',r} p(s', r|s, a) \left[r + \gamma V^*(s') \right]$$

This is the Bellman optimality equation.
Similarly:

$$Q^*(s, a) = \sum_{s',r} p(s', r|s, a) \left[r + \gamma \max_{a'} Q^*(s', a') \right]$$

Simple Numerical Example

Consider:

$$S_1 \to S_2 \to G$$

Suppose:

$$R(S_1 \to S_2) = -1$$

$$R(S_2 \to G) = 10$$

and:

$$\gamma = 0.9$$

Value of Goal

Since the goal is terminal:

$$V(G) = 0$$

Value of S_2

Using Bellman equation:

$$V(S_2) = 10 + 0.9(0)$$

thus:

$$V(S_2) = 10$$

Value of S_1

Now:

$$V(S_1) = -1 + 0.9(10)$$

$$V(S_1) = 8$$

Backup Interpretation

$$
\begin{array}{c}
V(S_1) \\
\downarrow \\
-1 \\
\downarrow \\
V(S_2) \\
\downarrow \\
10 \\
\downarrow \\
G
\end{array}
$$

The future reward information propagates backward from the goal toward earlier states.

This process is called a:

Bellman Backup

The Bellman equation emerges naturally from the recursive structure of future rewards.

The derivation follows these steps:

1. Define return
2. Observe recursive structure
3. Define value function
4. Substitute recursive return
5. Expand expectation using probabilities

The Bellman equation forms the mathematical foundation of:

- Dynamic Programming
- Temporal Difference Learning
- Q-Learning
- Policy Iteration
- Value Iteration
- Deep Reinforcement Learning

Dynamic Programming Methods

Dynamic Programming (DP) methods are classical reinforcement learning techniques used for solving Markov Decision Processes (MDPs).

DP methods assume:

- the environment model is completely known,
- transition probabilities are available,
- rewards are known.

The main idea is:

> Use Bellman equations repeatedly until value functions converge.

Policy Evaluation

Suppose a policy π is fixed.

The Bellman expectation equation is:

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma V^{\pi}(s') \right]$$

We iteratively update:

$$V_{k+1}(s) \leftarrow \sum_{a} \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma V_k(s') \right]$$

until convergence.

Value Iteration

Instead of evaluating a fixed policy, value iteration directly computes optimal values:

$$V_{k+1}(s) = \max_{a} \sum_{s',r} p(s', r|s, a) \left[r + \gamma V_k(s') \right]$$

The optimal policy is:

$$\pi^*(s) = \arg\max_{a} \sum_{s',r} p(s', r|s, a) \left[r + \gamma V^*(s') \right]$$

Dynamic Programming Algorithm

1. Initialize $V(s)$ arbitrarily
2. Repeat:
   - Update all states using Bellman equation
3. Stop when values converge

Worked Example

Consider:

$$S_1 \to S_2 \to G$$

Rewards:

$$R(S_1 \to S_2) = -1$$

$$R(S_2 \to G) = 10$$

Discount factor:

$$\gamma = 0.9$$

Initialize:

$$V_0(S_1) = 0, \qquad V_0(S_2) = 0$$

Iteration 1

$$V_1(S_2) = 10 + 0.9(0) = 10$$

$$V_1(S_1) = -1 + 0.9(0) = -1$$

Iteration 2

$$V_2(S_1) = -1 + 0.9(10) = 8$$

Thus:

$$V(S_1) = 8, \qquad V(S_2) = 10$$

Monte Carlo Methods

Monte Carlo (MC) methods learn value functions directly from experience.

Unlike DP:

- no environment model is needed,
- learning occurs from complete episodes.

Monte Carlo methods estimate values using sampled returns.

Monte Carlo State Value

The value function is:

$$V(s) = \mathbb{E}[G_t | S_t = s]$$

Monte Carlo estimates this by averaging observed returns.
Suppose state s is visited $N(s)$ times.
Then:

$$V(s) = \frac{\sum_{i=1}^{N(s)} G_i(s)}{N(s)}$$

where:

$$G_i(s)$$

is the observed return after visiting s.

Monte Carlo Algorithm

1. Generate complete episodes
2. Observe returns
3. Average returns for each state
4. Update value estimates

Worked Example

Suppose an agent experiences three episodes.

Episode 1

$$S_1 \to S_2 \to G$$

Rewards:

$$-1, \quad +10$$

Return from S_1:

$$G = -1 + 0.9(10) = 8$$

Return from S_2:

$$G = 10$$

Episode 2

$$S_1 \to S_2 \to G$$

Rewards again:

$$-1, \quad +10$$

Returns remain:

$$G(S_1) = 8, \qquad G(S_2) = 10$$

Monte Carlo Estimates

Average returns:

$$V(S_1) = \frac{8+8}{2} = 8$$

$$V(S_2) = \frac{10+10}{2} = 10$$

Thus:

$$V(S_1) = 8, \qquad V(S_2) = 10$$

Characteristics of Monte Carlo Methods

- Learn directly from experience
- Require complete episodes
- Unbiased estimates
- High variance

Temporal Difference Methods

Temporal Difference (TD) learning combines ideas from:

- Dynamic Programming
- Monte Carlo methods

TD learning updates estimates immediately after every step.
Unlike Monte Carlo:

- TD does not wait until episode termination.

Unlike DP:

- TD does not require a model.

TD(0) Update Rule

The Bellman equation suggests:

$$V(s) = \mathbb{E}[R_{t+1} + \gamma V(S_{t+1})]$$

TD learning updates:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right]$$

where:

- α is learning rate,
- term inside brackets is called TD error.

TD Error

The TD error is:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

It measures:

Difference between predicted value and improved estimate.

Temporal Difference Algorithm

1. Initialize $V(s)$
2. Observe transition:

$$S_t \to S_{t+1}$$

3. Update value immediately
4. Continue learning online

Worked Example

Again consider:

$$S_1 \to S_2 \to G$$

Rewards:

$$R(S_1 \to S_2) = -1$$

$$R(S_2 \to G) = 10$$

Discount factor:

$$\gamma = 0.9$$

Learning rate:

$$\alpha = 0.1$$

Initialize:

$$V(S_1) = 0, \qquad V(S_2) = 0$$

Update for S_2

Transition:

$$S_2 \to G$$

TD target:

$$10 + 0.9(0) = 10$$

Update:

$$V(S_2) = 0 + 0.1(10 - 0)$$

$$V(S_2) = 1$$

Update for S_1

Transition:

$$S_1 \to S_2$$

TD target:

$$-1 + 0.9(1) = -0.1$$

Update:

$$V(S_1) = 0 + 0.1(-0.1)$$

$$V(S_1) = -0.01$$

As more episodes occur, values gradually converge toward:

$$V(S_1) = 8, \qquad V(S_2) = 10$$

Comparison Between DP, MC and TD

Method	Model Needed	Episode Needed	Update Type
Dynamic Programming	Yes	No	Full backup
Monte Carlo	No	Yes	After episode
Temporal Difference	No	No	Step-by-step

- DP learns from complete environment models.
- MC learns from complete sampled episodes.
- TD learns by bootstrapping from current estimates.

Temporal Difference learning became one of the most important breakthroughs in reinforcement learning because it combines:

$$\text{Model-Free Learning} + \text{Online Updating} + \text{Bootstrapping}$$

which makes it scalable to large real-world problems.

SARSA and Q-Learning

SARSA and Q-Learning are two of the most important algorithms in reinforcement learning.

Both are:

- model-free reinforcement learning algorithms,
- temporal difference (TD) learning methods,
- based on learning action-value functions.

Their objective is to learn:

$$Q(s, a)$$

which represents:

Expected future reward obtained by taking action a in state s.

The major difference between them is:

SARSA is On-Policy

while

Q-Learning is Off-Policy

Action-Value Function

The action-value function is:

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

The optimal action-value function is:

$$Q^{*}(s,a) = \max_{\pi} Q^{\pi}(s,a)$$

Once $Q^{*}(s,a)$ is learned, the optimal policy becomes:

$$\pi^{*}(s) = \arg\max_{a} Q^{*}(s,a)$$

SARSA Algorithm

SARSA is named after the sequence:

$$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$$

used in its update rule.

SARSA Update Equation

The update rule is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

where:

- α is learning rate,
- γ is discount factor,
- term inside brackets is the TD error.

Interpretation

SARSA updates the current estimate using:

actual action chosen in next state

Thus the agent learns according to the policy it is actually following.

SARSA Algorithm Steps

1. Initialize $Q(s, a)$
2. Observe current state S_t
3. Choose action A_t using policy (e.g. ϵ-greedy)
4. Execute action
5. Observe:

$$R_{t+1}, S_{t+1}$$

6. Choose next action:

$$A_{t+1}$$

7. Update:

$$Q(S_t, A_t)$$

8. Repeat

Worked Example of SARSA

Suppose:

$$Q(S_1, \text{right}) = 2$$

Agent moves:

$$S_1 \to S_2$$

receives:

$$R = 5$$

Suppose next action selected is:

$$A_{t+1} = \text{up}$$

with:

$$Q(S_2, \text{up}) = 4$$

Let:

$$\alpha = 0.1, \qquad \gamma = 0.9$$

Update:

$$Q(S_1, \text{right}) = 2 + 0.1[5 + 0.9(4) - 2]$$

$$= 2 + 0.1(6.6)$$

$$= 2.66$$

Thus:

$$Q(S_1, \text{right}) = 2.66$$

Q-Learning Algorithm

Q-Learning is one of the most famous RL algorithms.

Unlike SARSA, Q-Learning learns the optimal policy directly.

Q-Learning Update Equation

The update rule is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Notice the key difference:

$$\max_a Q(S_{t+1}, a)$$

Instead of using the next action actually taken, Q-Learning assumes:

> The best possible action will always be selected in the future.

Q-Learning Algorithm Steps

1. Initialize $Q(s, a)$
2. Observe state S_t
3. Choose action A_t
4. Execute action
5. Observe:

$$R_{t+1}, S_{t+1}$$

6. Compute:

$$\max_a Q(S_{t+1}, a)$$

7. Update:

$$Q(S_t, A_t)$$

8. Repeat

Worked Example of Q-Learning

Suppose:

$$Q(S_1, \text{right}) = 2$$

Transition:

$$S_1 \to S_2$$

Reward:

$$R = 5$$

Suppose:

$$Q(S_2, \text{up}) = 4$$

$$Q(S_2, \text{down}) = 7$$

$$Q(S_2, \text{left}) = 3$$

Then:

$$\max_a Q(S_2, a) = 7$$

Using:

$$\alpha = 0.1, \qquad \gamma = 0.9$$

Update:

$$Q(S_1, \text{right}) = 2 + 0.1[5 + 0.9(7) - 2]$$

$$= 2 + 0.1(9.3)$$

$$= 2.93$$

Thus:

$$Q(S_1, \text{right}) = 2.93$$

Difference Between SARSA and Q-Learning

The critical difference is:

$$\boxed{\text{SARSA learns from the action actually taken}}$$

while

$$\boxed{\text{Q-Learning learns from the best possible action}}$$

Comparison Table

Feature	SARSA	Q-Learning
Type	On-policy	Off-policy
Update Uses	Actual next action	Best next action
Exploration Aware	Yes	No
Risk Behavior	Safer	More aggressive
Convergence	Slower	Often faster

On-Policy Methods

In **On-Policy** learning:

> The agent learns the value of the policy it is currently following.

The behavior policy and learning policy are the same.

SARSA is on-policy because it updates using the action actually chosen by the current policy.

Example

Suppose the agent follows an ϵ-greedy policy.

Sometimes it explores random actions.

SARSA learns including the effects of those exploratory actions.

Thus it learns the behavior of the actual policy being executed.

Off-Policy Methods

In **Off-Policy** learning:

> The agent learns about one policy while following another.

The behavior policy and target policy are different.
Q-Learning is off-policy because:

$$\max_a Q(S_{t+1}, a)$$

always assumes optimal future behavior even if the current behavior policy explores randomly.

Example

The agent may explore random moves during training.
However, Q-Learning updates as if the agent always behaves optimally in the future.
Thus:

$$\text{Behavior Policy} \neq \text{Target Policy}$$

Intuition

Suppose a robot walks near a cliff.

SARSA Behavior

SARSA considers exploratory actions.
Thus it learns safer paths because it accounts for accidental exploratory movements near dangerous regions.

Q-Learning Behavior

Q-Learning assumes future optimal behavior.
Thus it may learn aggressive shortest paths near cliffs because it assumes mistakes will not happen.

Backup Diagrams

SARSA Backup

$$Q(S_t, A_t) \to R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

Q-Learning Backup

$$Q(S_t, A_t) \to R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

Concept	Meaning
SARSA	On-policy TD control
Q-Learning	Off-policy TD control
On-policy	Learn current behavior policy
Off-policy	Learn optimal target policy
SARSA Update	Uses actual next action
Q-Learning Update	Uses maximum future action

SARSA and Q-Learning form the foundation of many modern reinforcement learning systems, including Deep Q-Networks (DQN) and several advanced deep reinforcement learning methods.

Policy Gradient Methods

So far, we have studied value-based reinforcement learning methods such as:

- Dynamic Programming
- SARSA
- Q-Learning

These methods first estimate value functions and then derive policies from them.

For example:

$$\pi(s) = \arg\max_a Q(s, a)$$

However, in many real-world problems:

- action spaces may be continuous,
- value estimation may become extremely difficult,
- deterministic policies may not be suitable.

In such situations, it is often better to directly parameterize and optimize the policy itself.

This leads to:

Policy Gradient Methods

Basic Idea

Instead of learning:

$$V(s) \quad \text{or} \quad Q(s,a)$$

we directly learn a parameterized policy:

$$\pi_\theta(a|s)$$

where:

- θ are policy parameters,
- usually represented using neural networks.

The objective is:

$$\max_\theta J(\theta)$$

where:

$$J(\theta)$$

is the expected return under policy π_θ.

Objective Function

The performance objective is defined as:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$$

where:

- τ denotes a trajectory,
- $G(\tau)$ is total reward of trajectory,
- trajectories are sampled using policy π_θ.

A trajectory is:

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots)$$

The goal is to compute:

$$\nabla_\theta J(\theta)$$

and perform gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Trajectory Probability

The probability of a trajectory under policy π_θ is:

$$P(\tau|\theta) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t)$$

where:

- $\rho_0(s_0)$ is initial state distribution,
- $P(s_{t+1}|s_t, a_t)$ are environment dynamics.

Thus:

$$J(\theta) = \sum_\tau P(\tau|\theta) G(\tau)$$

Derivation of Policy Gradient Theorem

We now derive the policy gradient theorem from scratch.

Step 1: Differentiate Objective

Starting with:

$$J(\theta) = \sum_\tau P(\tau|\theta) G(\tau)$$

Differentiate:

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau|\theta) G(\tau)$$

Since $G(\tau)$ does not depend directly on θ:

$$= \sum_\tau G(\tau) \nabla_\theta P(\tau|\theta)$$

Step 2: Log-Derivative Trick

Using the identity:

$$\nabla f(x) = f(x) \nabla \log f(x)$$

we obtain:

$$\nabla_\theta P(\tau|\theta) = P(\tau|\theta) \nabla_\theta \log P(\tau|\theta)$$

Substitute:

$$\nabla_\theta J(\theta) = \sum_\tau P(\tau|\theta) G(\tau) \nabla_\theta \log P(\tau|\theta)$$

which becomes:

$$= \mathbb{E}_{\tau \sim \pi_\theta} \left[G(\tau) \nabla_\theta \log P(\tau|\theta) \right]$$

Step 3: Expand Trajectory Log Probability

Recall:

$$P(\tau|\theta) = \rho_0(s_0) \prod_t \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t)$$

Taking logarithm:

$$\log P(\tau|\theta) = \log \rho_0(s_0) + \sum_t \log \pi_\theta(a_t|s_t) + \sum_t \log P(s_{t+1}|s_t, a_t)$$

Differentiate:

$$\nabla_\theta \log P(\tau|\theta) = \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)$$

because:

- initial distribution does not depend on θ,
- environment dynamics do not depend on θ.

Substitute back:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[G(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \right]$$

Rearranging:

$$= \mathbb{E} \left[\sum_t G(\tau) \nabla_\theta \log \pi_\theta(a_t|s_t) \right]$$

Policy Gradient Theorem

Thus the policy gradient theorem becomes:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) Q^{\pi_\theta}(s, a) \right]}$$

This is one of the most important equations in reinforcement learning.

Interpretation

The theorem says:

> Increase the probability of actions that produce high rewards.

If Action is Good

Suppose:

$$Q(s, a) > 0$$

Then:

$$\nabla_\theta \log \pi_\theta(a|s)$$

increases the probability of that action.

If Action is Bad

Suppose:

$$Q(s, a) < 0$$

Then the probability decreases.
Thus the policy gradually improves through gradient ascent.

REINFORCE Algorithm

One of the simplest policy gradient algorithms is:

REINFORCE

Update rule:

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(A_t|S_t)$$

Algorithm:

1. Generate episode
2. Compute returns G_t
3. Update policy parameters
4. Repeat

Worked Example

Suppose a policy chooses:

$$\{\text{left}, \text{right}\}$$

using softmax:

$$\pi_\theta(\text{right}|s) = \frac{e^\theta}{e^\theta + e^{-\theta}}$$

Suppose:

$$\theta = 0$$

Then:

$$\pi(\text{right}|s) = 0.5$$

Now assume:

- action "right" was selected,
- reward obtained:

$$G = 10$$

The gradient becomes:

$$\nabla_\theta \log \pi_\theta(\text{right}|s)$$

For softmax:

$$= 1 - \pi_\theta(\text{right}|s)$$

Thus:

$$= 1 - 0.5 = 0.5$$

Update:

$$\theta \leftarrow 0 + \alpha(10)(0.5)$$

If:

$$\alpha = 0.1$$

then:

$$\theta = 0.5$$

Now:

$$\pi(\text{right}|s) > 0.5$$

Thus the probability of successful actions increases.

Advantages of Policy Gradient Methods

- Naturally handle continuous action spaces
- Learn stochastic policies
- Better suited for high-dimensional problems
- Stable optimization in some settings

Disadvantages

- High variance gradients
- Sample inefficient
- Training can be unstable

Actor-Critic Methods

Modern RL often combines:

- policy gradients,
- and value estimation.

This leads to:

Actor-Critic Methods

where:

- Actor:

$$\pi_\theta(a|s)$$

  updates policy,

- Critic:

$$V(s) \quad \text{or} \quad Q(s,a)$$

  estimates value functions.

Algorithms such as:

- A2C
- A3C

- PPO
- DDPG
- SAC

are all built upon the policy gradient theorem.

Concept	Meaning
Policy Gradient	Direct policy optimization
$\pi_\theta(a\|s)$	Parameterized policy
Objective	Maximize expected reward
Gradient Method	Gradient ascent
REINFORCE	Basic policy gradient algorithm
Actor-Critic	Policy + Value learning

The policy gradient theorem forms the mathematical foundation of modern deep reinforcement learning methods.

REINFORCE Algorithm

REINFORCE is one of the simplest and most fundamental policy gradient algorithms.

It was introduced by Ronald Williams in 1992 and is based directly on the **Policy Gradient Theorem**.

The main idea is:

> Increase the probability of actions that lead to high rewards and decrease the probability of actions that lead to poor rewards.

Parameterized Policy

Suppose the policy is parameterized by:

$$\pi_\theta(a|s)$$

where:

- s is the state,
- a is the action,
- θ are policy parameters.

The objective is:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$$

We want:

$$\max_\theta J(\theta)$$

Policy Gradient Theorem

The policy gradient theorem states:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s) Q^\pi(s,a)\right]$$

In REINFORCE, we replace:

$$Q^\pi(s,a)$$

with sampled return:

$$G_t$$

Thus:

$$\nabla_\theta J(\theta) \approx G_t \nabla_\theta \log \pi_\theta(A_t|S_t)$$

REINFORCE Update Rule

The parameter update becomes:

$$\boxed{\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(A_t|S_t)}$$

where:

- α is learning rate,
- G_t is return from time step t.

Interpretation

Suppose:

$$G_t > 0$$

Then:

$$\log \pi_\theta(A_t|S_t)$$

is increased.
Hence:

$$\pi_\theta(A_t|S_t)$$

increases.
Thus good actions become more likely.
If:

$$G_t < 0$$

the probability decreases.

REINFORCE Algorithm

1. Initialize policy parameters θ
2. Generate an episode:

$$S_0, A_0, R_1, S_1, A_1, \ldots, S_T$$

3. For each time step:

$$G_t = R_{t+1} + \gamma R_{t+2} + \ldots$$

4. Update:

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(A_t|S_t)$$

5. Repeat

Worked Example of REINFORCE

Suppose a robot chooses:

$$\{\text{left}, \text{right}\}$$

using softmax policy:

$$\pi_\theta(\text{right}|s) = \frac{e^\theta}{e^\theta + e^{-\theta}}$$

Assume:

$$\theta = 0$$

Thus:

$$\pi(\text{right}|s) = 0.5$$

Suppose:

- action "right" was chosen,
- return:

$$G_t = 8$$

Gradient:

$$\nabla_\theta \log \pi_\theta(\text{right}|s) = 1 - \pi_\theta(\text{right}|s)$$

Thus:

$$= 1 - 0.5 = 0.5$$

Update:

$$\theta = 0 + 0.1(8)(0.5)$$

$$= 0.4$$

Now:

$$\pi(\text{right}|s) > 0.5$$

Thus successful actions become more probable.

Problem with REINFORCE

REINFORCE suffers from:

High Variance

because returns:

$$G_t$$

can fluctuate significantly.
This causes unstable learning.

REINFORCE with Baseline

To reduce variance, we introduce a baseline:

$$b(s)$$

The update becomes:

$$\boxed{\theta \leftarrow \theta + \alpha(G_t - b(S_t))\nabla_\theta \log \pi_\theta(A_t|S_t)}$$

Why Baseline Helps

Instead of using raw returns:

$$G_t$$

we compare rewards relative to expectation.

$$G_t - b(S_t)$$

measures:

> How much better or worse the action performed compared to normal expectation.

Common Baseline

Usually:

$$b(S_t) = V^{\pi}(S_t)$$

Thus:

$$G_t - V(S_t)$$

is called the:

Advantage Function

$$A(s, a) = Q(s, a) - V(s)$$

It measures:

How much better an action is than average

Advantage Interpretation

Suppose:

$$V(s) = 5$$

If:

$$G_t = 8$$

then:

$$A = 3$$

meaning the action was better than expected.
If:

$$G_t = 2$$

then:

$$A = -3$$

meaning the action was worse than expected.

REINFORCE with Baseline Algorithm

1. Generate episode
2. Compute returns G_t
3. Estimate baseline $V(S_t)$
4. Compute advantage:

$$A_t = G_t - V(S_t)$$

5. Update:

$$\theta \leftarrow \theta + \alpha A_t \nabla_\theta \log \pi_\theta(A_t|S_t)$$

Actor-Critic Methods

REINFORCE waits until episode termination.
This makes learning slow.
Actor-Critic methods improve this by combining:

- policy learning,
- value estimation,
- temporal difference learning.

Two Components

Actor

The actor learns the policy:

$$\pi_\theta(a|s)$$

It selects actions.

Critic

The critic estimates value functions:

$$V_w(s)$$

or:

$$Q_w(s, a)$$

where w are critic parameters.
The critic evaluates how good the actor's actions are.

Main Idea

$$\boxed{\text{Actor chooses actions}}$$

$$\boxed{\text{Critic evaluates actions}}$$

The critic provides learning signals to improve the actor.

Temporal Difference Error

The critic computes:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

called the TD error.
Interpretation:

- positive δ_t:

$$\text{better than expected}$$

- negative δ_t:

$$\text{worse than expected}$$

Actor Update

The actor updates:

$$\boxed{\theta \leftarrow \theta + \alpha \delta_t \nabla_\theta \log \pi_\theta(A_t|S_t)}$$

Critic Update

The critic updates value estimates:

$$\boxed{V(S_t) \leftarrow V(S_t) + \beta\delta_t}$$

where:

$$\beta$$

is critic learning rate.

Actor-Critic Algorithm

1. Observe state S_t
2. Actor selects action:

$$A_t \sim \pi_\theta(a|s)$$

3. Observe reward and next state
4. Compute TD error:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

5. Update actor
6. Update critic
7. Repeat

Worked Example

Suppose:

$$V(S_t) = 4$$

Reward:

$$R_{t+1} = 3$$

Next state value:

$$V(S_{t+1}) = 6$$

Discount:

$$\gamma = 0.9$$

Then:

$$\delta_t = 3 + 0.9(6) - 4$$

$$= 4.4$$

Positive TD error means:

$$\text{action was better than expected}$$

Thus actor increases probability of that action.

Advantages of Actor-Critic

- Lower variance than REINFORCE
- Online learning
- Faster convergence
- Works for continuous action spaces
- Scales well with deep learning

Disadvantages

- More complex
- Critic estimation errors may bias learning
- Training instability possible

Modern Deep RL Algorithms

Most modern reinforcement learning algorithms are based on Actor-Critic ideas:

- A2C
- A3C
- PPO
- DDPG
- TD3
- SAC

Method	Main Idea
REINFORCE	Pure policy gradient
REINFORCE + Baseline	Reduced variance
Actor-Critic	Policy + value learning
Actor	Learns policy
Critic	Learns value estimates
TD Error	Learning signal

REINFORCE introduced the idea of directly optimizing policies, while Actor-Critic methods made policy gradients practical and scalable for modern deep reinforcement learning.

Advantage Function

In reinforcement learning, the **advantage function** measures:

> How much better a particular action is compared to the average action at a given state.

The advantage function plays a central role in modern policy gradient methods such as:

- Actor-Critic
- PPO
- TRPO
- A2C / A3C

Motivation

Suppose the agent is in state:

$$s$$

and takes action:

$$a$$

The action-value function:

$$Q^{\pi}(s, a)$$

tells:

> Expected future return after taking action a.

The state-value function:

$$V^{\pi}(s)$$

tells:

Expected future return averaged over all actions under policy π.

Thus:

$$Q^{\pi}(s,a) - V^{\pi}(s)$$

measures whether action a is:

- better than average,
- worse than average,
- or average.

Definition of Advantage Function

The advantage function is defined as:

$$\boxed{A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)}$$

Interpretation

Positive Advantage

If:

$$A^{\pi}(s,a) > 0$$

then:

Action a is better than the average action in state s.

Negative Advantage

If:

$$A^{\pi}(s,a) < 0$$

then:

Action a is worse than expected.

Zero Advantage

If:

$$A^\pi(s, a) = 0$$

then the action is average under the policy.

Advantage in Policy Gradient

The policy gradient theorem becomes:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s) A^\pi(s, a)\right]$$

This improves learning because:

- good actions get reinforced,
- bad actions get suppressed,
- variance is reduced compared to raw returns.

Temporal Difference Error as Advantage Estimate

The one-step TD error is:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

Notice:

$$\delta_t \approx A^\pi(S_t, A_t)$$

Thus TD error acts as a simple estimate of advantage.

Problem with Simple TD Estimates

One-step TD estimates have:

- low variance,
- but high bias.

Monte Carlo returns have:

- low bias,
- but high variance.

We want a balance between:

$$\text{bias} \quad \text{and} \quad \text{variance}$$

This motivates:

Generalized Advantage Estimation (GAE)

Generalized Advantage Estimation (GAE)

GAE was introduced in:

Schulman et al. (2015)

The main idea is:

> Combine multi-step TD errors using exponential weighting.

Step 1: Define TD Residual

Define one-step TD residual:

$$\boxed{\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)}$$

This is the basic temporal difference error.

Step 2: Two-Step Advantage

Suppose we extend one-step estimation:

$$A_t^{(2)} = \delta_t + \gamma \delta_{t+1}$$

Expanding:

$$= R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) - V(S_t)$$

Similarly:

$$A_t^{(3)} = \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2}$$

Thus multi-step advantages progressively incorporate longer future information.

Generalized Advantage Estimation Formula

GAE combines all multi-step estimates:

$$\boxed{A_t^{GAE(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}}$$

where:

- γ is discount factor,
- λ controls bias-variance tradeoff.

Expanded Form

Expanding:

$$A_t^{GAE} = \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

Substituting TD errors:

$$= (R_{t+1} + \gamma V_{t+1} - V_t)$$

$$+\gamma\lambda(R_{t+2} + \gamma V_{t+2} - V_{t+1}) + \dots$$

This smoothly combines:

- short-term estimates,
- and long-term returns.

Special Cases

$\lambda = 0$

Then:

$$A_t^{GAE} = \delta_t$$

which becomes standard TD learning.
Low variance but high bias.

$\lambda = 1$

Then:

$$A_t^{GAE} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l}$$

which approaches Monte Carlo estimation.
Low bias but high variance.
Thus:

$$\lambda$$

controls interpolation between TD and Monte Carlo.

Derivation of GAE

Starting from:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

Define:

$$A_t^{GAE} = \delta_t + \gamma\lambda A_{t+1}^{GAE}$$

Recursively:

$$= \delta_t + \gamma\lambda(\delta_{t+1} + \gamma\lambda A_{t+2}^{GAE})$$

$$= \delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2 A_{t+2}^{GAE}$$

Continuing recursively:

$$= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

Thus GAE emerges naturally from exponentially weighted TD errors.

Worked Example

Suppose:

$$\gamma = 0.9, \qquad \lambda = 0.95$$

and TD errors are:

$$\delta_0 = 2$$

$$\delta_1 = 1$$

$$\delta_2 = 3$$

Compute:

$$A_0^{GAE} = 2 + (0.9)(0.95)(1) + (0.9 \times 0.95)^2(3)$$

First:

$$0.9 \times 0.95 = 0.855$$

Thus:

$$= 2 + 0.855 + (0.855)^2(3)$$

$$= 2 + 0.855 + 0.731(3)$$

$$= 2 + 0.855 + 2.193$$

$$= 5.048$$

Thus:

$$A_0^{GAE} \approx 5.05$$

Interpretation

Notice:

- nearby TD errors contribute strongly,
- distant TD errors contribute less.

This stabilizes learning while still preserving long-term information.

Why GAE Works Well

GAE became extremely important because it:

- reduces variance,
- maintains low bias,
- improves policy gradient stability,
- works extremely well with deep neural networks.

GAE in PPO and Modern RL

Modern algorithms like:

- PPO
- TRPO
- A2C
- A3C

almost always use GAE.
The policy update becomes:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) A_t^{GAE}]}$$

Thus GAE provides stable advantage estimates for deep reinforcement learning.

Concept	Meaning
$A(s,a)$	Relative goodness of action
TD Error	One-step advantage estimate
GAE	Weighted multi-step TD estimation
$\lambda = 0$	TD learning
$\lambda = 1$	Monte Carlo estimation
Purpose	Bias-variance balancing

Generalized Advantage Estimation is one of the most important developments in modern policy gradient reinforcement learning.

Trust Region Policy Optimization (TRPO)

Policy gradient methods directly optimize the policy:

$$\pi_\theta(a|s)$$

using gradient ascent.
However, ordinary policy gradient methods often suffer from:

- unstable updates,
- catastrophic policy collapse,
- excessively large parameter updates,
- sudden performance degradation.

The reason is:

A small change in parameters can produce a very large change in policy behavior.

TRPO (Trust Region Policy Optimization) was introduced to solve this problem.

The main idea is:

Improve policy while preventing excessively large updates

Core Idea of TRPO

Suppose:

$$\pi_{\theta_{old}}$$

is the old policy.

We want to obtain:

$$\pi_\theta$$

which improves performance.

Instead of maximizing reward directly, TRPO solves:

maximize performance improvement while staying close to old policy

Thus TRPO introduces:

- a surrogate objective,
- a trust-region constraint.

Trajectories Under Policies

A trajectory is:

$$\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$$

The old policy generates trajectories:

$$\tau \sim \pi_{\theta_{old}}$$

The new policy generates:

$$\tau \sim \pi_\theta$$

The objective is:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$$

But directly optimizing:

$$J(\theta)$$

is difficult because trajectory distributions themselves change when policy changes.

TRPO resolves this using importance sampling and surrogate objectives.

Performance Difference Lemma

Suppose:

$$\eta(\pi)$$

denotes expected discounted return:

$$\eta(\pi) = \mathbb{E}_{\tau \sim \pi}[G(\tau)]$$

Then:

$$\eta(\pi) = \eta(\pi_{old}) + \sum_{s} \rho_{\pi}(s) \sum_{a} \pi(a|s) A_{\pi_{old}}(s, a)$$

where:

- $\rho_{\pi}(s)$ is discounted state visitation frequency,
- $A_{\pi_{old}}(s, a)$ is advantage under old policy.

This equation says:

> Performance improvement depends on expected advantage under the new policy.

Problem

The visitation distribution:

$$\rho_{\pi}(s)$$

depends on the new policy.
This makes optimization difficult.

Local Approximation

TRPO approximates:

$$\rho_\pi(s) \approx \rho_{\pi_{old}}(s)$$

assuming policy updates are small.
Then:

$$\eta(\pi) \approx \eta(\pi_{old}) + \sum_s \rho_{\pi_{old}}(s) \sum_a \pi(a|s) A_{\pi_{old}}(s,a)$$

Now express:

$$\pi(a|s) = \frac{\pi(a|s)}{\pi_{old}(a|s)} \pi_{old}(a|s)$$

Substitute:

$$= \eta(\pi_{old}) + \sum_s \rho_{\pi_{old}}(s) \sum_a \pi_{old}(a|s) \frac{\pi(a|s)}{\pi_{old}(a|s)} A_{\pi_{old}}(s,a)$$

This becomes:

$$= \eta(\pi_{old}) + \mathbb{E}_{s,a\sim\pi_{old}} \left[\frac{\pi(a|s)}{\pi_{old}(a|s)} A_{\pi_{old}}(s,a) \right]$$

Surrogate Objective Function

Thus TRPO defines the surrogate objective:

$$\boxed{L_{\pi_{old}}(\pi) = \mathbb{E}_{s,a\sim\pi_{old}} \left[\frac{\pi(a|s)}{\pi_{old}(a|s)} A_{\pi_{old}}(s,a) \right]}$$

This is the central equation in TRPO.

Importance Sampling Ratio

Define:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

Interpretation:

- $r_t > 1$:

new policy increases probability

- $r_t < 1$:

$$\text{new policy decreases probability}$$

Then:

$$L(\theta) = \mathbb{E}[r_t(\theta)A_t]$$

Why Surrogate Objective Works

Trajectories are sampled using:

$$\pi_{old}$$

not the new policy.
Importance sampling corrects the mismatch:

$$\frac{\pi_\theta}{\pi_{old}}$$

Thus we can estimate performance improvement using old trajectories.

Problem with Large Updates

Suppose:

$$r_t(\theta)$$

becomes extremely large.
Then the new policy may differ dramatically from the old policy.
This can destabilize learning.
Thus TRPO constrains policy updates.

KL Divergence Constraint

TRPO restricts policy change using KL divergence.
The constraint is:

$$\boxed{\mathbb{E}[D_{KL}(\pi_{old}(\cdot|s)||\pi_\theta(\cdot|s))] \leq \delta}$$

where:

- D_{KL} measures policy difference,
- δ controls trust-region size.

Final TRPO Optimization Problem

Thus TRPO solves:

$$\boxed{\max_{\theta} \mathbb{E}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t\right]}$$

subject to:

$$\boxed{\mathbb{E}[D_{KL}(\pi_{\theta_{old}}||\pi_\theta)] \leq \delta}$$

Interpretation

TRPO says:

> Improve policy performance while remaining close to the old policy.

This creates stable monotonic policy improvement.

Geometric Interpretation

Ordinary gradient ascent:

$$\theta_{new} = \theta_{old} + \alpha \nabla_\theta J$$

may jump too far in parameter space.
TRPO instead constrains updates inside a:

trust region

where local approximations remain valid.

Natural Gradient

TRPO approximates constrained optimization using the natural gradient.
The Fisher Information Matrix is:

$$F = \mathbb{E}[\nabla_\theta \log \pi_\theta \nabla_\theta \log \pi_\theta^T]$$

The natural gradient direction is:

$$\boxed{F^{-1} g}$$

where:

$$g = \nabla_\theta L(\theta)$$

Unlike ordinary gradients, natural gradients account for geometry of probability distributions.

TRPO Algorithm

1. Initialize policy:

$$\pi_{\theta_{old}}$$

2. Collect trajectories using old policy
3. Estimate:

$$A_t$$

   using GAE or TD methods

4. Compute surrogate objective:

$$L(\theta) = \mathbb{E}[r_t(\theta)A_t]$$

5. Compute policy gradient:

$$g = \nabla_\theta L(\theta)$$

6. Compute Fisher matrix:

$$F$$

7. Solve:

$$F^{-1}g$$

   using conjugate gradient

8. Perform constrained line search satisfying KL constraint
9. Update policy:

$$\theta_{new}$$

10. Repeat

Worked Example

Suppose:

$$\pi_{old}(a|s) = 0.4$$

New policy proposes:

$$\pi_\theta(a|s) = 0.6$$

Advantage:

$$A_t = 5$$

Importance ratio:

$$r_t = \frac{0.6}{0.4} = 1.5$$

Surrogate objective contribution:

$$r_t A_t = 1.5(5) = 7.5$$

Thus increasing probability of this good action improves objective.
Now suppose KL divergence becomes too large.
Then TRPO rejects overly aggressive updates and reduces step size.

Why TRPO Became Important

TRPO introduced:

- stable policy optimization,
- monotonic improvement guarantees,
- trust-region constrained updates,
- natural gradient optimization.

It became the foundation for:

- PPO
- advanced actor-critic methods
- modern policy optimization

Limitations of TRPO

Although powerful, TRPO has drawbacks:

- second-order optimization is expensive,
- Fisher matrix inversion is computationally heavy,
- implementation complexity.

These limitations motivated PPO.

Concept	Meaning
Old Policy	$\pi_{\theta_{old}}$
New Policy	π_θ
Surrogate Objective	Stable performance estimate
Importance Ratio	$\frac{\pi_\theta}{\pi_{old}}$
Trust Region	KL divergence constraint
Natural Gradient	Geometry-aware optimization
Goal	Stable policy improvement

TRPO was one of the first algorithms that made deep policy optimization stable and practical for large-scale reinforcement learning problems.

Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is one of the most successful and widely used reinforcement learning algorithms.

It was introduced by OpenAI in 2017 as a simpler alternative to TRPO.

PPO became extremely popular because it provides:

- stable learning,
- simple implementation,
- high performance,
- scalability with deep neural networks.

Modern reinforcement learning systems frequently use PPO for:

- robotics,
- game playing,
- large language model alignment,
- RLHF,
- autonomous control.

Motivation

Ordinary policy gradient methods perform updates:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

However, large updates can cause:

- unstable policies,
- catastrophic collapse,
- sudden performance drops.

TRPO solved this using a KL divergence constraint:

$$D_{KL}(\pi_{old} || \pi_\theta)$$

but TRPO requires:

- second-order optimization,
- Fisher matrix estimation,
- conjugate gradient methods.

These are computationally expensive.
PPO was designed to approximate TRPO while remaining simple.

Core Idea of PPO

The central idea is:

$$\boxed{\text{Prevent excessively large policy updates}}$$

Instead of hard constraints like TRPO, PPO uses:

$$\textbf{clipped probability ratios}$$

to softly restrict policy updates.

Policy Ratio

Suppose:

$$\pi_{\theta_{old}}$$

is the old policy.
We update to:

$$\pi_\theta$$

Define the probability ratio:

$$\boxed{r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}}$$

Interpretation:

- $r_t > 1$:

  new policy increases action probability

- $r_t < 1$:

  new policy decreases action probability

Policy Gradient Objective

Ordinary policy gradient maximizes:

$$L^{PG}(\theta) = \mathbb{E}[r_t(\theta)A_t]$$

where:

$$A_t$$

is the advantage estimate.
Problem:

$$r_t(\theta)$$

may become very large.
This causes unstable updates.

PPO Clipped Objective

PPO introduces clipping:

$$\boxed{L^{CLIP}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right]}$$

This is the central equation of PPO.

Understanding the Clipping

Suppose:

$$\epsilon = 0.2$$

Then:

$$r_t(\theta) \in [0.8, 1.2]$$

is considered acceptable.
If the ratio exceeds this range, PPO clips it.

Positive Advantage Case

Suppose:

$$A_t > 0$$

meaning the action is good.
Then PPO wants to increase:

$$\pi_\theta(a_t|s_t)$$

However, PPO prevents:

$$r_t(\theta)$$

from becoming too large.
Thus:

$$r_t(\theta) \leq 1 + \epsilon$$

Negative Advantage Case

Suppose:

$$A_t < 0$$

meaning the action is bad.
PPO wants to reduce its probability.
But PPO prevents drastic reduction:

$$r_t(\theta) \geq 1 - \epsilon$$

Thus updates remain conservative.

Why the Minimum Operator?

The PPO objective uses:

$$\min(\cdot, \cdot)$$

because PPO always chooses the pessimistic lower bound.
This prevents the optimizer from exploiting large probability changes.
Thus PPO stabilizes training.

Trajectory Collection

Suppose the old policy:

$$\pi_{\theta_{old}}$$

generates trajectories:

$$\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$$

Using these trajectories we compute:

- returns,
- value estimates,
- advantages.

Usually advantages are computed using:

Generalized Advantage Estimation (GAE)

Advantage Estimation

The TD error is:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

GAE computes:

$$A_t^{GAE} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

This balances:

- bias,
- variance.

Value Function Loss

PPO also trains a critic network.

The value loss is:

$$\boxed{L^{VF} = (V_\phi(s_t) - V_t^{target})^2}$$

where:

- V_ϕ is critic,
- V_t^{target} is target return.

Entropy Bonus

PPO often adds entropy regularization:

$$\boxed{L^{ENT} = -\sum_a \pi_\theta(a|s) \log \pi_\theta(a|s)}$$

Purpose:

- encourage exploration,
- avoid premature convergence.

Final PPO Objective

The total PPO loss becomes:

$$\boxed{L^{TOTAL} = L^{CLIP} - c_1 L^{VF} + c_2 L^{ENT}}$$

where:

- c_1 controls critic loss,
- c_2 controls entropy bonus.

PPO Algorithm

1. Initialize actor and critic networks
2. Collect trajectories using old policy
3. Compute:
    - returns,
    - advantages using GAE

4. Compute probability ratio:

$$r_t(\theta)$$

5. Compute clipped objective
6. Update actor using gradient ascent
7. Update critic using value loss
8. Repeat for multiple epochs

Worked Example

Suppose:

$$\pi_{old}(a|s) = 0.4$$

New policy predicts:

$$\pi_\theta(a|s) = 0.6$$

Then:

$$r_t = \frac{0.6}{0.4} = 1.5$$

Suppose:

$$A_t = 4$$

Ordinary objective:

$$r_t A_t = 1.5(4) = 6$$

Now let:

$$\epsilon = 0.2$$

Then clipped ratio:

$$\text{clip}(1.5, 0.8, 1.2) = 1.2$$

Clipped objective:

$$1.2(4) = 4.8$$

PPO chooses:

$$\min(6, 4.8) = 4.8$$

Thus PPO suppresses overly aggressive updates.

Geometric Interpretation

PPO creates a soft trust region around the old policy:

$$\pi_{\theta_{old}}$$

Instead of strict KL constraints like TRPO, PPO softly penalizes large changes through clipping.

Thus PPO approximates trust-region optimization using only first-order gradients.

Why PPO Works So Well

PPO became extremely successful because it combines:

- stability,
- simplicity,
- sample efficiency,
- scalability.

Unlike TRPO:

- no Fisher matrix,
- no conjugate gradients,
- no second-order optimization.

Actor-Critic Structure in PPO

PPO usually uses Actor-Critic architecture.

Actor

Learns policy:

$$\pi_\theta(a|s)$$

Critic

Learns value function:

$$V_\phi(s)$$

PPO in Large Language Models

Modern RLHF systems frequently use PPO.

The language model acts as:

$$\pi_\theta$$

Human feedback provides reward signals.

PPO updates the language model while preventing catastrophic changes in behavior.

Advantages of PPO

- Stable learning
- Simple implementation
- First-order optimization only
- Works with large neural networks
- Strong empirical performance

Limitations

- Still sample inefficient
- Sensitive to hyperparameters
- Clipping may sometimes overly restrict learning

Concept	Meaning
PPO	Stable policy optimization
Core Idea	Restrict large updates
Ratio	$\frac{\pi_\theta}{\pi_{old}}$
Clipping	Prevent policy collapse
GAE	Advantage estimation
Actor	Policy network
Critic	Value network
Entropy Bonus	Encourage exploration

PPO is currently one of the most important and widely used reinforcement learning algorithms in modern AI systems.

Direct Preference Optimization (DPO)

Direct Preference Optimization (DPO) is a reinforcement learning style optimization method used for aligning large language models with human preferences.

It was introduced as a simpler alternative to:

RLHF (Reinforcement Learning from Human Feedback)

especially avoiding the complexity of PPO-based optimization.
DPO became important because it removes the need for:

- explicit reward model training,
- policy rollouts,
- reinforcement learning optimization loops,
- value functions,
- actor-critic architectures.

Instead, DPO directly optimizes the language model using preference data.

Motivation

Suppose a language model produces two responses:

$$y_w \quad \text{(preferred/winner)}$$

and

$$y_l \quad \text{(less preferred/loser)}$$

Human annotators choose:

$$y_w \succ y_l$$

meaning:

winner response preferred over loser response

Traditional RLHF proceeds in multiple stages:

1. Supervised Fine-Tuning (SFT)
2. Train reward model
3. PPO optimization using reward model

This pipeline is complicated and unstable.
DPO eliminates the reward model explicitly.

Core Idea of DPO

The main insight is:

> If humans prefer one response over another, then the optimal policy should assign higher probability to the preferred response.

Thus instead of learning rewards explicitly, DPO directly adjusts policy probabilities.

Preference Dataset

The dataset consists of tuples:

$$(x, y_w, y_l)$$

where:

- x is prompt,
- y_w is preferred response,
- y_l is rejected response.

Example:

$$x : \text{“Explain gravity”}$$

Preferred response:

$$y_w : \text{clear scientific explanation}$$

Rejected response:

$$y_l : \text{incorrect confusing explanation}$$

RLHF Objective

In RLHF, the optimal policy solves:

$$\boxed{\max_{\pi} \mathbb{E}_{y\sim\pi}[r(x,y)] - \beta D_{KL}(\pi||\pi_{ref})}$$

where:

- $r(x, y)$ is reward model,
- π_{ref} is reference model,
- KL term prevents policy drift.

DPO derives an equivalent objective without explicitly training:

$$r(x, y)$$

Optimal Policy in RLHF

The optimal policy for KL-constrained RL is:

$$\boxed{\pi^*(y|x) \propto \pi_{ref}(y|x) \exp\left(\frac{1}{\beta} r(x,y)\right)}$$

Taking logarithm:

$$\log \pi^*(y|x) = \log \pi_{ref}(y|x) + \frac{1}{\beta} r(x,y) + C$$

Rearranging:

$$\boxed{r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{ref}(y|x)} + C}$$

This is the key insight of DPO.

Preference Probability

Suppose humans choose preferred response according to Bradley-Terry model:

$$P(y_w \succ y_l) = \frac{\exp(r(x,y_w))}{\exp(r(x,y_w)) + \exp(r(x,y_l))}$$

Substitute reward expression:

$$r(x,y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$$

Then:

$$P(y_w \succ y_l) = \frac{\exp\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}\right)}{\exp\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}\right) + \exp\left(\beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)}$$

Using:

$$e^{\log x} = x$$

we get:

$$= \frac{\left(\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}\right)^\beta}{\left(\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}\right)^\beta + \left(\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)^\beta}$$

DPO Loss Function

The final DPO objective becomes:

$$\boxed{L_{DPO} = -\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} \right)}$$

where:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

is the sigmoid function.

Simplified Form

Combining logarithms:

$$\boxed{L_{DPO} = -\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)\pi_{ref}(y_l|x)}{\pi_\theta(y_l|x)\pi_{ref}(y_w|x)} \right)}$$

This is the standard DPO loss.

Interpretation

DPO increases probability of preferred responses:

$$y_w$$

and decreases probability of rejected responses:

$$y_l$$

relative to the reference model.

Role of Reference Model

The reference model:

$$\pi_{ref}$$

acts as an anchor.
It prevents catastrophic drift from the original pretrained model.
Thus DPO implicitly contains KL regularization.

Geometric Interpretation

Suppose:

$$\pi_\theta(y_w|x)$$

increases.
Then:

$$L_{DPO} \downarrow$$

Thus optimization encourages:

$$\pi_\theta(y_w|x) > \pi_\theta(y_l|x)$$

while remaining close to:

$$\pi_{ref}$$

Worked Example

Suppose:

$$\pi_\theta(y_w|x) = 0.6$$

$$\pi_\theta(y_l|x) = 0.2$$

Reference model:

$$\pi_{ref}(y_w|x) = 0.5$$

$$\pi_{ref}(y_l|x) = 0.3$$

Let:

$$\beta = 1$$

Compute:

$$z = \log \frac{0.6}{0.5} - \log \frac{0.2}{0.3}$$

$$= \log(1.2) - \log(0.667)$$

$$= 0.182 - (-0.405)$$

$$= 0.587$$

Sigmoid:

$$\sigma(0.587) = 0.642$$

Loss:

$$L_{DPO} = -\log(0.642)$$

$$= 0.443$$

Thus optimization will further increase probability of preferred response.

DPO Algorithm

1. Start with pretrained reference model:

$$\pi_{ref}$$

2. Collect preference pairs:

$$(x, y_w, y_l)$$

3. Initialize trainable policy:

$$\pi_\theta$$

4. Compute DPO loss
5. Update model using gradient descent
6. Repeat

Difference Between PPO and DPO

Feature	PPO	DPO
Reward Model	Required	Not needed
RL Optimization	Yes	No
Value Function	Needed	Not needed
Sampling Rollouts	Required	Not required
Complexity	High	Lower
Training Stability	Difficult	Simpler

Advantages of DPO

- Simpler than PPO
- Stable optimization
- No reward model training
- No RL rollouts
- Easy implementation
- Efficient large-scale training

Limitations

- Requires preference datasets
- Less theoretically general than RL
- Depends strongly on quality of preference annotations

Why DPO Became Important

DPO became influential because it demonstrated:

> Preference alignment can be achieved without explicit reinforcement learning.

It simplified alignment training pipelines for large language models.

Applications

DPO is widely used for:

- LLM alignment,
- chatbot preference tuning,
- instruction following,
- harmlessness and safety tuning,
- conversational AI.

Concept	Meaning
DPO	Direct Preference Optimization
Goal	Align policy with preferences
Input Data	Preferred/rejected response pairs
Reference Model	Stability anchor
Main Idea	Increase preferred response probability
RL Needed?	No explicit RL loop
Reward Model Needed?	No

DPO provides a mathematically elegant and computationally efficient alternative to PPO-based RLHF for aligning large language models with human preferences.

Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization (GRPO) is a reinforcement learning based optimization method designed primarily for large language model reasoning and alignment.

GRPO became popular through its use in reasoning-oriented LLM training because it removes the need for:

- explicit critic networks,
- separate value function training,
- complicated advantage estimation models.

Instead, GRPO computes advantages **relative to a group of sampled responses**.

The core idea is:

Learn by comparing responses within the same sampled group

GRPO can be viewed as a simplified actor-only variant of PPO.

Motivation

In PPO, policy optimization requires:

- actor network,
- critic network,
- value estimation,
- GAE computation.

However, critic training is often:

- unstable,
- memory expensive,
- difficult for long reasoning tasks.

GRPO avoids critic learning entirely.
Instead of estimating:

$$V(s)$$

GRPO computes rewards relative to a sampled group.

Basic Setup

Suppose a prompt is:

$$x$$

The current policy:

$$\pi_\theta(y|x)$$

generates multiple responses:

$$\{y_1, y_2, \ldots, y_G\}$$

where:

$$G$$

is the group size.
Each response receives a reward:

$$r_i = r(x, y_i)$$

using:

- reward models,
- correctness signals,
- rule-based evaluation,
- human preferences.

Core Idea of GRPO

Instead of using an explicit value function baseline:

$$V(s)$$

GRPO computes:

group-relative advantages

using statistics of the sampled group itself.

Group Mean Reward

Compute mean reward:

$$\boxed{\bar{r} = \frac{1}{G}\sum_{i=1}^{G} r_i}$$

This represents average performance of the sampled group.

Group Relative Advantage

The advantage of sample i becomes:

$$\boxed{A_i = r_i - \bar{r}}$$

Interpretation:

- positive advantage:

  $$r_i > \bar{r}$$

  means better-than-average response,

- negative advantage:

  $$r_i < \bar{r}$$

  means worse-than-average response.

Thus learning becomes purely comparative.

Normalized Advantages

Often advantages are normalized:

$$\boxed{A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}}$$

where:

- σ_r is reward standard deviation,
- ϵ prevents division by zero.

This stabilizes optimization.

Connection to PPO

Recall PPO objective:

$$L^{PPO} = \mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t)]$$

GRPO uses the same clipped policy optimization idea.
The difference is:

GRPO computes advantages using group comparisons instead of critics

Probability Ratio

Suppose:

$$\pi_{\theta_{old}}$$

is old policy.
Define probability ratio:

$$\boxed{r_i(\theta) = \frac{\pi_\theta(y_i|x)}{\pi_{\theta_{old}}(y_i|x)}}$$

This measures how much the new policy changes probability of sampled response.

GRPO Objective

The GRPO objective becomes:

$$\boxed{L^{GRPO} = \mathbb{E}\left[\min\left(r_i(\theta)A_i, \text{clip}(r_i(\theta), 1-\epsilon, 1+\epsilon)A_i\right)\right]}$$

Notice:

- same PPO clipping,
- but no value function.

Why Relative Rewards Work

Suppose all responses are poor.
Absolute rewards may still be noisy.
But relative comparison tells:

Which response is better among the generated candidates.

This becomes especially useful for reasoning tasks.

Reasoning Example

Suppose prompt:

$$x : \text{“What is } 25\times 18\text{?”}$$

Model generates 4 responses:

$$y_1 = 450$$

$$y_2 = 430$$

$$y_3 = 250$$

$$y_4 = 460$$

Suppose rewards:

$$r_1 = 1, \quad r_2 = 0, \quad r_3 = 0, \quad r_4 = 0$$

Mean reward:

$$\bar{r} = \frac{1+0+0+0}{4} = 0.25$$

Advantages:

$$A_1 = 0.75$$

$$A_2 = -0.25$$

$$A_3 = -0.25$$

$$A_4 = -0.25$$

Thus GRPO increases probability of:

$$y_1$$

while suppressing incorrect outputs.

Trajectory Interpretation

In PPO, trajectories often correspond to:

$$(s_t, a_t, r_t)$$

In language models:

$$\text{trajectory} = (x, y)$$

where:

- x is prompt,
- y is generated response sequence.

GRPO samples multiple trajectories for the same prompt and compares them internally.

Why GRPO Removes Critic

Traditional actor-critic methods require learning:

$$V(s)$$

which approximates expected future reward.
GRPO bypasses this by using:

$$\bar{r}$$

as an implicit baseline.

Thus:

$$A_i = r_i - \bar{r}$$

acts similarly to:

$$A(s,a) = Q(s,a) - V(s)$$

without explicitly learning:

$$V(s)$$

KL Regularization

GRPO usually includes KL penalties against reference policy:

$$\pi_{ref}$$

The objective becomes:

$$\boxed{L = L^{GRPO} - \beta D_{KL}(\pi_\theta || \pi_{ref})}$$

Purpose:

- prevent policy collapse,
- avoid catastrophic drift,
- preserve pretrained capabilities.

GRPO Algorithm

1. Initialize policy:

$$\pi_\theta$$

2. Sample prompt:

$$x$$

3. Generate group of responses:

$$\{y_1, \ldots, y_G\}$$

4. Compute rewards:

$$r_i$$

5. Compute group mean:

$$\bar{r}$$

6. Compute advantages:

$$A_i = r_i - \bar{r}$$

7. Compute PPO-style clipped objective
8. Update policy parameters
9. Repeat

Comparison with PPO

Feature	PPO	GRPO
Critic Network	Required	Not required
Value Function	Needed	Not needed
Advantage Estimation	GAE/TD	Group relative rewards
Complexity	Higher	Lower
Memory Usage	Higher	Lower
Reasoning Tasks	Good	Very effective

Advantages of GRPO

- Simpler than PPO
- No critic training
- Lower memory usage
- Stable optimization
- Effective for reasoning LLMs
- Naturally comparative learning

Limitations

- Requires multiple generations per prompt
- Computationally expensive sampling
- Relative rewards may ignore absolute quality
- Depends strongly on reward quality

Why GRPO Works Well for Reasoning

Reasoning tasks naturally involve:

- multiple candidate solutions,
- internal comparison,
- selecting best reasoning path.

GRPO directly optimizes this comparative structure.
Thus it works extremely well for:

- mathematical reasoning,
- coding,
- chain-of-thought generation,
- logical inference.

Connection to Human Learning

Humans often learn relatively:

> This answer is better than that answer.

rather than estimating exact numerical values.
GRPO mimics this comparative learning process.

Concept	Meaning
GRPO	Group Relative Policy Optimization
Core Idea	Relative comparison learning
Advantage	$r_i - \bar{r}$
Critic Needed?	No
Optimization	PPO-style clipping
Main Benefit	Simpler stable RL for LLMs

GRPO represents an important evolution of PPO-style reinforcement learning for large language models by replacing explicit critic estimation with efficient group-relative comparisons.

A Unified Toy Model for Reinforcement Learning Algorithms

In this section we construct a very simple environment which can be used to understand almost all major reinforcement learning algorithms:

- Dynamic Programming
- Monte Carlo
- Temporal Difference Learning
- SARSA
- Q-Learning
- REINFORCE
- REINFORCE with Baseline
- Actor-Critic
- PPO
- DPO
- GRPO

The purpose is not to solve a complicated environment but to understand how each algorithm fundamentally operates.

Environment Setup

Consider a simple 1D world:

$$S_1 \to S_2 \to S_3 \to G$$

where:

- S_1, S_2, S_3 are states,
- G is goal state.

At each state the agent can choose:

$$A = \{\text{Left}, \text{Right}\}$$

The transition rules are:

Current State	Action	Next State
S_1	Right	S_2
S_1	Left	S_1
S_2	Right	S_3
S_2	Left	S_1
S_3	Right	G
S_3	Left	S_2

Reward Structure

The reward function is:

$$R = \begin{cases} +10 & \text{if goal reached} \\ -1 & \text{otherwise} \end{cases}$$

Thus:

- reaching goal is highly rewarding,
- wandering gives penalties.

Markov Decision Process Formulation

The MDP is:

$$(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$$

where:

- States:

$$\mathcal{S} = \{S_1, S_2, S_3, G\}$$

- Actions:

$$\mathcal{A} = \{L, R\}$$

- Transition probabilities:

$$P(s'|s, a)$$

  deterministic here.

- Rewards:

$$R(s, a, s')$$

- Discount:

$$\gamma = 0.9$$

1. Dynamic Programming

Suppose the model is fully known.

We use Bellman equation:

$$V(s) = \max_a \sum_{s'} P(s'|s,a)[R + \gamma V(s')]$$

For S_3:

$$V(S_3) = 10 + \gamma V(G)$$

Assume:

$$V(G) = 0$$

Then:

$$V(S_3) = 10$$

For S_2:

$$V(S_2) = -1 + 0.9(10)$$

$$= 8$$

For S_1:

$$V(S_1) = -1 + 0.9(8)$$

$$= 6.2$$

Thus DP computes exact optimal values.

2. Monte Carlo Method

Now assume model is unknown.

Suppose episode:

$$S_1 \to S_2 \to S_3 \to G$$

Rewards:

$$-1, -1, +10$$

Return from S_1:

$$G_0 = -1 + 0.9(-1) + 0.9^2(10)$$

$$= -1 - 0.9 + 8.1$$

$$= 6.2$$

Monte Carlo estimates:

$$V(S_1) \approx 6.2$$

using complete episodes.

3. Temporal Difference Learning

TD updates before episode termination.

Suppose:

$$V(S_2) = 8$$

Observed transition:

$$S_1 \to S_2$$

Reward:

$$R = -1$$

Update:

$$V(S_1) \leftarrow V(S_1) + \alpha[-1 + 0.9(8) - V(S_1)]$$

TD bootstraps using current estimates.

4. SARSA

SARSA learns action values.

Suppose:

$$Q(S_1, R) = 2$$

Agent moves right and next chosen action is also right:

$$Q(S_2, R) = 5$$

Update:

$$Q(S_1, R) \leftarrow Q(S_1, R) + \alpha[-1 + 0.9(5) - 2]$$

SARSA uses:

$$\text{actual next action}$$

Thus it is on-policy.

5. Q-Learning

Suppose:

$$Q(S_2, R) = 5$$

$$Q(S_2, L) = 2$$

Update:

$$Q(S_1, R) \leftarrow Q(S_1, R) + \alpha[-1 + 0.9 \max(5, 2) - 2]$$

Q-Learning assumes future optimal actions.
Thus it is off-policy.

6. REINFORCE

Now instead of learning values, we directly learn policy:

$$\pi_\theta(a|s)$$

Suppose:

$$\pi_\theta(R|S_1) = 0.6$$

Trajectory:

$$S_1 \to S_2 \to S_3 \to G$$

Return:

$$G_0 = 6.2$$

Update:

$$\theta \leftarrow \theta + \alpha G_0 \nabla_\theta \log \pi_\theta(R|S_1)$$

Thus successful actions become more likely.

7. REINFORCE with Baseline

Suppose baseline:

$$V(S_1) = 5$$

Advantage:

$$A = 6.2 - 5 = 1.2$$

Update:

$$\theta \leftarrow \theta + \alpha(1.2)\nabla_\theta \log \pi_\theta(R|S_1)$$

Instead of raw return, we use relative improvement.
This reduces variance.

8. Actor-Critic

Actor:

$$\pi_\theta(a|s)$$

Critic:

$$V_w(s)$$

Suppose:

$$V(S_1) = 5$$

Observed:

$$R = -1, \quad V(S_2) = 8$$

TD error:

$$\delta = -1 + 0.9(8) - 5$$

$$= 1.2$$

Critic update:

$$V(S_1) \leftarrow V(S_1) + \beta\delta$$

Actor update:

$$\theta \leftarrow \theta + \alpha\delta\nabla_\theta \log \pi_\theta(R|S_1)$$

9. PPO

Suppose old policy:

$$\pi_{old}(R|S_1) = 0.5$$

New policy:

$$\pi_\theta(R|S_1) = 0.8$$

Ratio:

$$r = \frac{0.8}{0.5} = 1.6$$

Suppose:

$$A = 2$$

PPO objective:

$$L = \min(1.6(2), 1.2(2))$$

assuming:

$$\epsilon = 0.2$$

Thus:

$$L = 2.4$$

instead of:

$$3.2$$

PPO clips excessively large updates.

10. DPO

Suppose prompt:

$$x : \text{“Choose best path”}$$

Preferred response:

$$y_w : \text{“Move Right repeatedly”}$$

Rejected response:

$$y_l : \text{“Move Left repeatedly”}$$

Policy probabilities:

$$\pi_\theta(y_w|x) = 0.7$$

$$\pi_\theta(y_l|x) = 0.2$$

Reference model:

$$\pi_{ref}(y_w|x) = 0.5$$

$$\pi_{ref}(y_l|x) = 0.3$$

DPO increases probability of:

$$y_w$$

relative to:

$$y_l$$

without explicit RL rollouts.

11. GRPO

Suppose prompt:

$$x : \text{“Find shortest route to goal”}$$

Generated responses:

$$y_1 : \text{Right Right Right}$$

$$y_2 : \text{Left Left}$$

$$y_3 : \text{Right Left Right}$$

Rewards:

$$r_1 = 10, \quad r_2 = -5, \quad r_3 = 2$$

Mean reward:

$$\bar{r} = \frac{10 - 5 + 2}{3} = 2.33$$

Advantages:

$$A_1 = 7.67$$

$$A_2 = -7.33$$

$$A_3 = -0.33$$

GRPO increases probability of:

$$y_1$$

relative to the others.

Unified Interpretation

All algorithms attempt to solve:

How should the agent act to maximize future rewards?

But they differ in:

- whether model is known,
- whether values or policies are learned,
- how rewards are estimated,
- how optimization is stabilized.

Algorithm	Main Idea
Dynamic Programming	Exact Bellman optimization
Monte Carlo	Learn from complete episodes
TD Learning	Bootstrap future estimates
SARSA	On-policy action learning
Q-Learning	Off-policy optimal learning
REINFORCE	Direct policy gradient
REINFORCE+Baseline	Variance reduction
Actor-Critic	Policy + value learning
PPO	Stable clipped policy optimization
DPO	Preference-based direct optimization
GRPO	Relative group-based optimization

Evolution of RL Algorithms

The historical evolution can be viewed as:

$$\text{DP} \rightarrow \text{MC} \rightarrow \text{TD} \rightarrow \text{Q-Learning}$$

$$\rightarrow \text{Policy Gradients} \rightarrow \text{Actor-Critic} \rightarrow \text{PPO}$$

$$\rightarrow \text{DPO/GRPO}$$

Modern large language model alignment methods are deeply connected to classical reinforcement learning ideas developed over decades.